

Aprovado no benchmark, reprovado no README: vazamento e aprendizado por atalho em um classificador de prompt injection em português do Brasil

Rodrigo Faria

Pesquisador independente

<https://github.com/alucardigo> · <https://www.linkedin.com/in/faria-rodriigo>

8 de outubro de 2026

Resumo

Classificadores de prompt injection estão cada vez mais entre agentes de LLM e o conteúdo que eles buscam. Relatamos um estudo de caso totalmente rastreado da construção de um classificador desse tipo para o português do Brasil (pt-BR) com dados abertos. Nosso melhor modelo, um multilingual-e5-large com fine-tuning, detectou 94,7% das injeções do Weni, um conjunto de teste nativo em pt-BR, mas o número não sobreviveu à descontaminação: retreinada sem duplicatas nem quase duplicatas dos testes, a mesma receita detecta 68,0%. Pontuando página a página pelo caminho de código de um hook de agente, as duas versões deram aviso falso em todas as 217 páginas de documentação técnica, em todo limiar de 0,5 a 0,999. Entre cinco classificadores públicos, um também avisou em toda página; só o Llama Prompt Guard 2 (86M) manteve o aviso falso abaixo de 2% com detecção útil (1,8% e 62%; o de 22M não avisou em nenhuma, mas detectou 15%); e os benchmarks por frase e por página ordenaram os modelos em sentidos opostos. Acrescentar 10.000 textos benignos técnicos sustentou o nosso diagnóstico, um atalho que equipara prosa técnica a injeção: os avisos falsos sumiram nas frases técnicas e caíram para 19–35% das páginas, enquanto a detecção no Weni ficou perto da do modelo descontaminado (63,7% contra 68,0%). Nenhuma versão que treinamos atende aos dois critérios de aceitação. Contribuímos com o estudo de caso, uma auditoria de vazamento conferida por retreino, um protocolo de avaliação por página, lições de integração com agentes e artefatos abertos.

1 Introdução

Agentes de LLM leem texto não confiável: páginas web, saídas de ferramentas, arquivos de repositório, e-mails. Qualquer parte disso pode carregar instruções endereçadas ao modelo, e não ao usuário. A prompt injection foi demonstrada primeiro por entrada direta do usuário (Pérez and Ribeiro, 2022); quando as instruções chegam dentro de conteúdo que a aplicação recupera, ela é conhecida como prompt injection indireta (Greshake et al., 2023; Liu et al., 2024). Uma mitigação comum é um classificador pequeno que pontua o conteúdo buscado e avisa, ou bloqueia, quando ele parece uma injeção (Jacob et al., 2025; Meta, 2025; ProtectAI.com, 2024). Esses classificadores costumam ser relatados com acurácia por frase em benchmarks públicos, quase todos em inglês.

Este artigo documenta o que aconteceu quando construímos um para o português do Brasil, usando apenas dados abertos, e tentamos ligá-lo como hook de aviso na frente da ferramenta de web fetch de um agente de programação baseado em LLM. O núcleo do estudo levou uma semana (30 de setembro a 6 de outubro de 2026); um baseline com o Llama Prompt Guard 2 e um retreino

descontaminado vieram em 7 de outubro. Cada número abaixo é rastreado até uma linha de log, um arquivo de resultado ou um commit em um dossiê de evidências.

O resultado principal é negativo, em duas partes. O melhor modelo passou nos testes de benchmark com folga, errou por um único texto o próprio critério de aceitação, fixado de antemão, no conjunto ouro nativo em pt-BR (284/300 contra 285 exigidos), e depois, avaliado página a página em documentação técnica comum, deu aviso falso em 217 de 217 páginas em todo limiar que testamos. Uma frase de teste de fumaça escrita à mão vinha sinalizando o modo de falha desde a segunda rodada de treino, e não a lemos como o sinal que ela era (Seção 6). Depois, o próprio número do benchmark se mostrou inflado. Uma auditoria a posteriori tinha estimado em cerca de 1,3 ponto o efeito das quase duplicatas do conjunto ouro nos dados de treino; retrainar a mesma receita sem duplicatas exatas de nenhum item de teste, sem as traduções delas, sem quase duplicatas do conjunto ouro, sem as linhas que um conjunto agregado tinha copiado do xTRam1 e sem as excluídas pela licença (v3-pub) baixou a detecção no Weni para 68,0%, 26,7 pontos a menos nos mesmos itens, enquanto a falha por página ficou exatamente igual (Seção 4, Tabela 5). A queda limita a inflação por cima, em vez de medi-la: a v3-pub também tem menos exemplos de ataque, e ela cai mais nos textos do Weni sem nenhuma quase duplicata nos dados de treino da v3 (29,8 pontos) do que nos que têm uma (13,8).

Outras duas medições tornam o resultado mais nítido. Sob o mesmo protocolo de página, a falha não é exclusiva do nosso modelo: um de cinco classificadores públicos também avisou em toda página, e outros dois avisaram em cerca de 15% das páginas benignas no limiar 0,5, detectando 63–70% das páginas com injeção (Seção 5.4). O caso instrutivo é o Llama Prompt Guard 2, cujo card relata dados benignos da web no treino dos dois tamanhos. No limiar 0,5, o de 86M avisou em 4 das 217 páginas benignas (1,8%) e detectou 62% das páginas com injeção, embora tenha pegado só 30% das injeções curtas em inglês e detecte 93,7% das frases do Weni, abaixo do nosso critério; o de 22M não avisou em nenhuma página, mas detectou só 15% das páginas com injeção. E o retraining com texto benigno técnico (v6, Seção 7) eliminou os avisos falsos em frases técnicas e reduziu os avisos falsos por página para 19–35%. Ele também baixou a detecção no Weni dos 94,7% da v3, inflados por vazamento, para 63,7%, o que lemos primeiro como o preço de tirar o atalho. A v3-pub descontaminada, que não tem negativos técnicos e continua avisando em toda página, cai quase o mesmo: a v6 fica 4,3 pontos abaixo dela, sem significância, então a comparação com a v3 não permite cobrar essa queda dos negativos novos. As duas, porém, erram textos em parte diferentes, e quanto da queda da v6 os negativos novos causam continua em aberto; a melhora por página, ao contrário, é deles (Seção 7.2). Nenhum dos nossos modelos atende aos dois critérios de aceitação. Entre os modelos públicos, só o Prompt Guard 2 (86M) atende ao critério por página no limiar 0,5 detectando uma parte útil das páginas com injeção, e só na estimativa pontual (intervalo de 95% de 0,7–4,6%); o de 22M atende a ele com folga (0 de 217 páginas; intervalo de 0–1,7%), mas detecta pouco demais para servir de guard.

Achamos que o caso é útil além do pt-BR porque a falha não é exótica. O lado benigno dos dados de treino era em grande parte conversacional (comandos a assistentes e prompts de seguimento de instruções); a distribuição de implantação é sobretudo prosa técnica cheia de imperativos (instale, rode, configure, ignore este aviso). Um classificador pode separar injeções de conversa com acurácia quase perfeita e, ainda assim, não ter aprendido nada que separe injeções de documentação. O vazamento também não é exótico: ele entrou por cópias traduzidas automaticamente de itens de teste e por conjuntos agregados que contêm outros benchmarks, e uma auditoria por similaridade de embeddings contra o conjunto ouro não viu a maior parte do quanto o placar dependia dos dados vazados.

Um estudo concorrente de Liu (2026), publicado enquanto este trabalho estava em andamento, reavalia quinze detectores em inglês sobre saídas de ferramentas de agentes e sobre trechos benignos

de READMEs do GitHub, e chega às mesmas conclusões práticas. Lemos o nosso estudo como uma replicação independente dessa lacuna em outra língua e com um detector treinado por nós, o que permite rastrear a causa até a receita de treino e testá-la por retreino; ele acrescenta a agregação por página inteira, lições de integração para hooks de agentes e a perspectiva dos modelos de decisão tipados (Seção 9.3).

Contribuições.

1. **Um estudo de caso documentado de uma lacuna entre benchmark e uso em pt-BR.** Nove rodadas de treino (sondas sobre embeddings congelados; e5-base e e5-large com fine-tuning, com e sem dados de ataque adicionais; um retreino descontaminado; e um retreino com texto benigno técnico que testa o diagnóstico) mais um ensemble de pesos fixos, avaliados em até 19 conjuntos de teste, com um critério de aceitação fixado no controle de versão antes da primeira medição no conjunto ouro, e uma auditoria de vazamento conferida por retreino: um corte de similaridade de cosseno tinha estimado em cerca de 1,3 ponto a inflação dos 94,7% do melhor modelo, e a mesma receita retreinada sem duplicatas nem quase duplicatas dos testes detecta 68,0%.
2. **Um protocolo de avaliação por página para classificadores de injeção** que reutiliza exatamente o caminho de código do hook de implantação, pontua páginas pelo máximo sobre janelas sobrepostas e relata avisos falsos em páginas técnicas benignas ao lado da detecção em páginas com uma injeção inserida. Aplicado a cinco classificadores públicos, ele mostra que as avaliações por frase e por página podem ordenar os modelos em sentidos opostos. O benchmark é distribuído como um manifesto fixado por hash (217 páginas benignas e 60 inserções) mais um construtor que baixa as páginas das fontes, de modo que nenhum texto de terceiros é redistribuído.
3. **Lições de engenharia para integrar classificadores a agentes:** um bug de encoding que fazia texto benigno parecer injeção, uma incompatibilidade de versão do servidor engolida em silêncio por um hook fail-open, uma janela deslizante mais longa que o contexto do modelo e uma lista de entradas de avaliação que mudou entre execuções.
4. **Artefatos abertos:** código (Apache-2.0), inclusive o construtor do benchmark de páginas, e todos os arquivos de métricas e o manifesto do benchmark (CC-BY-4.0), publicados com este artigo; os pesos da v3-pub (MIT) são publicados à parte (Seção 13).

O restante do artigo está organizado assim. A Seção 2 apresenta a fundamentação sobre modelos de decisão tipados e sobre prompt injection. A Seção 3 descreve dados, tradução, controle de vazamento e treino. A Seção 4 relata os resultados por frase e a auditoria de vazamento, e a Seção 5, os resultados por página, com baselines públicos. A Seção 6 diagnostica a lacuna, a Seção 7 testa o diagnóstico por retreino, a Seção 8 lista as lições de integração, a Seção 9 discute o que isso diz sobre a acurácia de benchmark, sobre o uso de modelos de decisão com agentes e sobre trabalhos concorrentes, a Seção 10 lista as limitações e a Seção 11 conclui.

2 Fundamentação

2.1 Modelos de decisão tipados (“System One”)

Uma linha recente de produtos e modelos abertos responde a perguntas *fechadas* sobre texto: quem chama fornece um texto e um conjunto fixo de opções, e o modelo devolve a opção escolhida com uma probabilidade, sem gerar prosa. A TypeSafe divulga assim o seu modelo hospedado Jev, como

um modelo “System One”, nome que o post de lançamento atribui à distinção de Kahneman entre o pensamento rápido e intuitivo e o pensamento lento e deliberado (Almeida, 2026; Kahneman, 2011). Em 1º de outubro de 2026 a Cloudflare lançou o Clef e o Clef-flash, modelos de decisão de pesos abertos (Apache-2.0) que aceitam a mesma API (Chen et al., 2026; Cloudflare, 2026). O Kev-9B é outro modelo de decisão de pesos abertos, um adaptador LoRA e uma cabeça de decisão sobre um modelo base de 9B (Palmer, 2026). O Laya, da Convai Innovations, é uma implementação local aberta (Apache-2.0) de decisões tipadas (escolha única, nota e saídas binárias) que roda em CPU e foi feita para receber fine-tuning no domínio do próprio usuário (Convai Innovations, 2026). O Decision Index é uma suíte pública de benchmarks para essa família de modelos; ele não redistribui seus dados de teste e, em vez disso, distribui construtores que reproduzem os arquivos byte a byte e recusam qualquer arquivo cujo hash não confira (apolinario, 2026). Nesse placar, o checkpoint zero-shot do Laya aparece muito abaixo do Kev-9B, da classe de 9B (apolinario, 2026).

Um post em português de Fabio Akita, escrito para desenvolvedores, discute esses modelos e compara o Jev com o Clef e o Laya (Akita, 2026a). Segue a nossa paráfrase dos pontos relevantes aqui. Akita trata o Jev como um serviço de classificação na nuvem, competente e de baixo custo, e não como uma nova categoria de modelo, e sustenta que o ganho de velocidade anunciado chama atenção apenas porque a base de comparação são modelos de chat, que redigem um texto antes que se possa extrair o rótulo. Boa parte do post é um ensaio escrito para ele por Paulo Câmara, que declara ter cofundado uma empresa que concorre no mesmo mercado. Refazendo as contas de testes públicos de terceiros, o ensaio relata que o Jev fica com a opção que vem em primeiro lugar, e com confiança alta, quando não há nada no texto que permita decidir, e que suas decisões variam com atributos que não têm relação com o caso. Na visão de Akita, a ferramenta convém a decisões que se repetem sobre texto, com desfechos conhecidos de antemão, em que o código determinístico em volta do modelo garante as regras rígidas e um erro tem como ser flagrado ou passar por conferência. Entre os conselhos de uso do ensaio estão ajustar de novo a calibração sobre uma amostra modesta de casos rotulados pelo próprio usuário, e só então definir o limiar, e não confiar só ao modelo a liberação de operações destrutivas. Um post anterior do mesmo autor argumentou que o remédio, para equipes que pedem rótulos a modelos de chat, não passa obrigatoriamente pela contratação de um serviço desses: pedir ao modelo já adotado uma resposta em formato estruturado, ou treinar um classificador dedicado àquela tarefa, também atende (Akita, 2026b).

Este estudo é um exemplo da segunda opção: um classificador binário dedicado (“este conteúdo buscado é uma injeção?”) treinado para um único trabalho. Ele foi implantado como um *pack* de decisão em um kit de ferramentas local construído em torno do servidor aberto do Laya; o classificador em si é um modelo multilingual-e5 com fine-tuning e não deriva dos pesos do Laya.

2.2 Prompt injection e seus detectores

A prompt injection foi descrita primeiro para entrada direta do usuário (Pérez and Ribeiro, 2022) e depois para conteúdo que uma aplicação recupera em nome do usuário (Greshake et al., 2023); seguiram-se tratamentos formais e benchmarks (Debenedetti et al., 2024; Liu et al., 2024; Yi et al., 2025; Zhan et al., 2024). A competição HackAPrompt produziu um grande corpus público de prompts adversariais escritos por humanos, rotulados conforme tiveram sucesso ou não (Schulhoff et al., 2023). As defesas vão da separação entre instruções e dados, seja marcando a entrada não confiável dentro do prompt (Hines et al., 2024), seja treinando o modelo em um formato de consulta estruturado (Chen et al., 2025), a desenhos de sistema que não deixam dados não confiáveis dirigirem o fluxo de controle (Debenedetti et al., 2025), e a detectores: classificadores DeBERTa com fine-tuning (deepset, 2023a; ProtectAI.com, 2024), a família Prompt Guard da Meta (Meta, 2024, 2025), entre outros. O excesso de defesa (over-defence), isto é, entradas benignas marcadas

porque compartilham traços de superfície com ataques, foi documentado em modelos de guarda em inglês e atribuído a um viés em direção a palavras-gatilho (Li and Liu, 2024; Li et al., 2025). Dois estudos recentes são próximos do nosso: um benchmark multieixo conclui que o aumento de dados com negativos difíceis elimina o excesso de defesa em entradas benignas curadas, mas o deixa em grande parte intacto nas obtidas de fontes externas (Shire and Kim, 2026), e uma reavaliação de quinze detectores públicos sobre saídas de ferramentas de agentes relata que dois deles marcaram todos os trechos benignos de README do GitHub que pontuaram (Liu, 2026) (comparamos com ele na Seção 9.3). A avaliação em taxas baixas de falso positivo tem sido defendida como o regime relevante para a implantação (Jacob et al., 2025; Liu, 2026).

A nossa falha é conhecida em aprendizado de máquina sob outro nome: aprendizado por atalho (shortcut learning) (Geirhos et al., 2020), em que um modelo explora um traço que separa as classes na distribuição de treino, mas não na de implantação, como nos artefatos de anotação e nas heurísticas sintáticas em NLI (Gururangan et al., 2018; McCoy et al., 2019), e nas quedas de acurácia observadas em conjuntos de teste recém-coletados e sob mudança de distribuição no mundo real (Koh et al., 2021; Recht et al., 2019). O teste comportamental além da acurácia em dados separados (Ribeiro et al., 2020) é o remédio geral; a Seção 5 o aplica, para detectores de injeção, na unidade de uso.

Recursos para pt-BR são escassos. O único conjunto nativo de injeções em pt-BR de tamanho utilizável que encontramos é o do Weni (Weni, 2024); existe um conjunto multilíngue com itens em português (rikka-snow, 2024). A maior parte dos nossos dados é em inglês, o que motivou o pipeline de tradução automática descrito abaixo.

3 Método

3.1 Dados

A Tabela 1 lista cada fonte de dados, sua licença declarada, seu papel e seu tamanho. Os tamanhos por fonte são diferenças entre contagens acumuladas consecutivas nos logs de treino. Quando um conjunto tem split `test` oficial, nós o usamos; caso contrário, o conjunto de teste é o bucket de hash ≥ 80 de uma partição por md5, limitado a 1.000 itens.

Referências dos conjuntos de dados: deepset (deepset, 2023b), jackhhao (jackhhao, 2023), SPML (Sharma, 2024; Sharma et al., 2024), xTRam1 (xTRam1, 2024), rikka (rikka-snow, 2024), ShieldLM (dmilush, 2026), yanis (Miraoui, 2023), MASSIVE (FitzGerald et al., 2023), Weni (Weni, 2024), Dolly (Conover et al., 2023), HackAPrompt (Schulhoff et al., 2023) via o espelho (imoxto, 2023), Stack Overflow (koutch, 2023; pacovaldez, 2022), CodeSearchNet (Husain et al., 2019; Nan-Do, 2023).

Dois testes compostos são usados ao longo do texto. *pt-native* junta o Weni (300 positivos) e o MASSIVE-pt (1.500 negativos), 1.800 itens. *Fumaça* são dez frases escritas no código, cinco injeções e cinco benignas; uma das frases benignas é uma instrução de instalação em português (“Instruções de instalação: execute o setup.exe e clique em avançar.”), que acaba importando (Seção 6).

Dois fatos descobertos depois das rodadas de treino mudam o modo como os testes devem ser lidos. Primeiro, **o xTRam1 não está fora da distribuição** para nenhum modelo treinado com o ShieldLM: 577 das 4.000 linhas de treino do ShieldLM vêm do próprio split de treino do xTRam1, e 4 dos 1.500 itens de teste do xTRam1 aparecem no subconjunto de treino do ShieldLM (correspondência exata após normalização de caixa e de espaços). Uma conferência posterior achou mais sobreposição (abaixo). Por isso descrevemos o xTRam1 como “mesma distribuição, itens em parte sobrepostos” para todo modelo anterior à v3-pub (Seção 3.4). Mesmo a v3-pub, treinada sem essas linhas e sem nenhuma duplicata dos 1.500 itens de teste do xTRam1, mantém 422 textos

Tabela 1: Fontes de dados. “MT” = cópias traduzidas por máquina acrescentadas por cima (Seção 3.2).

Fonte (id no Hugging Face)	Licença declarada	Papel	Treino	Teste
deepset/prompt-injections (en/de)	Apache-2.0	treino	546	116
jackhhao/jailbreak-classification	Apache-2.0	treino	1.044	262
reshabhs/SPML_Chatbot_Prompt_Injection	MIT	treino (subamostra)	4.000	1.000
xTRam1/safe-guard-prompt-injection	nenhuma declarada	só teste	0	1.500
rikka-snow/prompt-injection-multilingual (pt)	MIT	treino (a partir da rodada B)	6.006	1.276
dmilush/shieldlm-prompt-injection	MIT (curadoria)	treino (a partir da B)	4.000	4.000
yanismiraoui/prompt_injections (só positivos)	Apache-2.0 + NOTICE	treino (a partir da B)	815	219
mteb/amazon_massive_intent (pt), benigno	CC-BY-4.0 (origem)	treino (a partir da B)	1.500	1.500
Weni/prompt-injections-1.0.0 (pt-BR, só injeções)	nenhuma declarada	só teste ouro	0	300
databricks/databricks-dolly-15k, benigno	CC-BY-SA-3.0	treino (a partir da v2)	3.000	1.000
HackAPrompt (espelho, só ataques bem-sucedidos)	nenhuma no espelho; origem MIT	treino (v4, v5)	3.000	1.000
Perguntas / respostas do Stack Overflow; docstrings do CodeSearchNet, benignos técnicos	CC-BY-SA (SO); por repositório (CSN)	treino (só v6)	4.000 / 4.000 / 2.000	1.000 cada

de treino, 378 deles do jackhhao e 37 da Dolly, idênticos a textos de outras partes do conjunto xTRam1, então o xTRam1 também não é inteiramente externo para ela. O Weni é o único teste genuinamente externo. Segundo, 14 dos 262 itens de teste do jackhhao aparecem no subconjunto de treino do ShieldLM; o deepset não tem nenhum item nesse subconjunto (0/116). Uma conferência de todo o conjunto de treino da v3 contra todos os testes, feita para a v3-pub (Seção 3.4), achou duplicatas exatas, após normalização de caixa e de espaços, em quase todo teste: 20 dos 262 itens do jackhhao, 72 dos 1.000 do SPML, 83 dos 1.500 do xTRam1, 439 dos 4.000 do ShieldLM, 75 dos 1.500 do MASSIVE-pt e algumas nos demais; nenhuma no deepset nem no Weni. Além disso, a configuração em português do MASSIVE é português europeu, não brasileiro.

3.2 Tradução para pt-BR

Os dados de treino e de teste em inglês foram traduzidos localmente com `Helsinki-NLP/opus-mt-tc-big-en-pt`, um modelo OPUS-MT treinado no Tatoeba Translation Challenge (Helsinki-NLP, 2022; Tiedemann, 2020; Tiedemann and Thottingal, 2020), token de idioma `>>por<<`, decodificação gulosa (um feixe, sem amostragem), o que torna a tradução determinística. As entradas são truncadas em 512 tokens e 1.500 caracteres. Até 1.000 exemplos por fonte em inglês (os de hash par) são acrescentados ao treino, e os primeiros 300 itens de teste do SPML e 500 do xTRam1 são traduzidos como testes adicionais; os testes do deepset e do jackhhao são traduzidos por inteiro. A rodada A usou uma regra anterior: metade dos dados de treino por hash, com um teto único de 3.000.

O teto de geração foi fixado originalmente em 512 tokens novos; a decodificação gulosa caía em repetição e corria até o teto, cerca de 7 s por instrução curta. Limitar a geração a $\min(512; 1,6 \times \text{tokens de entrada} + 16)$ eliminou o problema; o ganho de velocidade de 5–10× relatado está registrado apenas em uma mensagem de commit e não foi medido de forma independente (Seção 10). As traduções ficam em cache pelo SHA-1 do texto-fonte completo e são gravadas em disco a cada lote, e o trabalho foi dividido entre duas máquinas com CPU; a cota de GPU foi reservada para o fine-tuning. A rodada A traduziu 2.723 textos de treino e 1.161 de teste. Nenhum texto traduzido foi revisado por um humano.

3.3 Partição e protocolo de seleção

Todas as partições são funções determinísticas do texto: `bucket = int(md5(text)[:8], 16) mod 100`. O conjunto de validação da sonda é `bucket < 15` (15%). Para as sondas, a constante de regularização $C \in \{0,05; 0,2; 1; 4\}$ e o peso de classe $\in \{\text{nenhuma}, \text{balanceado}\}$ são escolhidos pela acurácia balanceada na validação, e uma temperatura é ajustada na validação minimizando a log-loss sobre uma grade logarítmica de 60 pontos de 0,1 a cerca de 20. Os conjuntos de teste são usados só para medir. Os conjuntos de teste nunca são enviados ao serviço de fine-tuning: a exportação para o fine-tuning na nuvem contém apenas dados de treino, e a avaliação roda nas máquinas do próprio autor (um laptop e uma VM ARM na nuvem).

Os critérios de aceitação foram fixados antes da medição (Tabela 2). O critério usado na decisão ($\text{Weni} \geq 95\%$ e $\text{xTRam1-pt} \geq 90\%$) foi registrado em commit em 2 de outubro, quase três dias antes da primeira medição no Weni. O critério por página só foi acrescentado depois da falha por página e vale para rodadas futuras.

3.4 Trava anti-vazamento

Acredita-se que o conjunto Weni seja uma tradução para o português de material do HackAPrompt; a afirmação aparece no nosso código e no histórico de commits, mas não foi verificada de forma

Tabela 2: Critérios de aceitação.

Data	Critério	Situação quando fixado
1 out	$\geq 95\%$ no xTRam1, então considerado fora da distribuição (Seção 3.1)	antes dos resultados da rodada 0
2 out	Weni $\geq 95\%$ e xTRam1-pt $\geq 90\%$; se atendido, ligar um hook só de aviso e fail-open	antes de qualquer medição no Weni
6 out	taxa de aviso falso por página $< 2\%$	depois da falha por página; para rodadas futuras

independente na época. Para manter honesto o teste ouro, a partir da rodada v4 é removido todo exemplo de treino cuja similaridade de cosseno máxima com qualquer texto do Weni passe de 0,9 (multilingual-e5-base, vetores normalizados). Na exportação da v4 isso removeu 4.008 de 27.693 exemplos. O HackAPrompt entra apenas pelas linhas rotuladas como ataques bem-sucedidos; do contrário, um ataque fracassado viraria um exemplo benigno implícito.

A trava não foi aplicada à exportação v2/v3, que é anterior a ela. Aplicar depois a mesma regra aos dados de treino da v3 marca 115 exemplos como quase duplicatas de textos do Weni. Eles são em sua maioria traduções automáticas (98 injeções e 17 exemplos benignos; 48 do jackhhao-pt, 18 do SPML-pt e 10 cada do deepset-pt e do Dolly-pt), e 58 dos 300 textos do Weni têm um deles como vizinho. Voltamos a isso nas Seções 4, 7 e 10.

Retreino descontaminado (v3-pub). Em 7 de outubro retreinamos a receita da v3 sobre um subconjunto estrito dos seus dados de treino, tanto para medir quanto do seu placar no Weni dependia de exemplos vazados quanto para obter pesos livres do problema de licença da Seção 12. Quatro filtros, aplicados em ordem, removem 1.758 dos 23.693 exemplos e não acrescentam nada: 673 linhas cujo texto vem de fontes do ShieldLM que são o próprio xTRam1 (as 577 linhas da Seção 3.1) ou que têm licença diferente na origem e no agregador (653 linhas do ShieldLM e 20 cópias dos mesmos textos no jackhhao e na Dolly); 831 textos idênticos, após normalização de caixa e de espaços, a algum texto de qualquer teste, inclusive os traduzidos; 164 traduções cujo original saiu, ou originais cuja tradução saiu; e 90 exemplos acima de cosseno 0,9 com algum texto do Weni, a trava acima (os outros 25 dos 115 já tinham saído nos filtros anteriores). O resultado tem 21.935 exemplos, 38,6% deles injeções (39,8% na v3). Das 967 injeções removidas, 513 são duplicatas exatas de itens de teste, 256 vêm das fontes excluídas do ShieldLM e 198 são pares de tradução ou quase duplicatas do Weni. Depois dos filtros, nenhum item de teste tem duplicata normalizada no treino, e nenhum texto de treino é idêntico a, compartilha um trecho de 200 caracteres com ou compartilha um 13-grama com algum dos 334 READMEs de pacotes de onde saiu o teste de página. Receita, notebook, código de avaliação, máquina, testes e páginas congeladas são os da v3 e da v6; a acurácia na validação interna foi de 99,49%.

3.5 Modelos e treino

Sondas (rodadas 0, A, B). `intfloat/multilingual-e5-base` congelado (intfloat, 2023; Wang et al., 2024) com o prefixo `query:` e limite de 256 tokens, seguido de uma regressão logística padronizada. Os embeddings foram calculados em uma CPU de laptop (Intel i7-1255U).

Modelos com fine-tuning (v2–v5). Classificação binária de sequência sobre `multilingual-e5-base` ou `multilingual-e5-large`, entrada `query:` + texto, 256 tokens no treino e na inferência, 3 épocas, taxa de aprendizado $2e-5$, weight decay 0,01, aquecimento (warm-up) em 6% dos passos, fp16, melhor época pela acurácia de validação em um split estratificado de 8% com semente 0. O e5-base usou batch 32; o e5-large usou batch 8 com 4 passos de acumulação e gradient check-

Tabela 3: Rodadas de treino.

Rodada	Data	Modelo	Dados de treino	Tamanho	Nota
0	1 out	sonda, e5-base	deepset + jackhhao + SPML, só inglês	5.590 exemplos; 4.622 após o split de validação	—
A	2–5 out	sonda, e5-base	rodada 0 + 2.723 MT pt	n_train 6.910	tradução
B	5 out	sonda, e5-base	7 fontes incl. pt nativo, 17.911 + MT	n_train 17.580	dados pt nativos
v2	5 out	e5-base com fine-tuning	20.911 originais (8 fontes de treino: as 7 da rodada B + Dolly) + 2.782 MT = 23.693	val. 1.896	sem trava anti-vazamento
v3	5 out	e5-large com fine-tuning	mesma exportação da v2	val. 1.896	sem trava anti-vazamento
v4	6 out	e5-base com fine-tuning	+ HackAPrompt; a trava removeu 4.008 → 23.685	val. 1.895	trava anti-vazamento
v5	6 out	e5-large com fine-tuning	exportação da v4	val. 1.895	trava anti-vazamento
mistura	6 out	logits v3 $\oplus$ v5, 0,5/0,5	—	—	sem ajuste no teste
v6	6 out	e5-large com fine-tuning	exportação da v3 – 115 quase duplicatas do Weni + 10.000 benignos técnicos	33.578 exemplos	trava anti-vazamento; negativos técnicos (Seção 7)
v3-pub	7 out	e5-large com fine-tuning	exportação da v3 – 1.758 (Seção 3.4)	21.935 exemplos	descontaminada contra todos os testes

pointing. O treino rodou em um notebook gratuito do Kaggle com duas GPUs NVIDIA T4 (15.360 MiB cada); se as duas GPUs foram de fato usadas não está verificado (Seção 10). **Na inferência os modelos com fine-tuning usam temperatura 1,0, isto é, logits crus sem calibração.**

Ensemble de pesos fixos. Os logits da v3 e da v5 são combinados por média com pesos 0,5/0,5, escolhidos antes de olhar qualquer resultado de teste, porque não existe um conjunto de validação separado do Weni e ajustar o peso no teste burlaria o critério.

3.6 Rodadas

A Tabela 3 resume as rodadas. A maioria das rodadas mudou um fator principal em relação a uma rodada de referência; três não: a v2 passou para fine-tuning e acrescentou o Dolly como dado benigno ao mesmo tempo, a v4/v5 acrescentam tanto o HackAPrompt quanto a trava anti-vazamento em relação à v2/v3, e a v6 acrescenta negativos técnicos e tira as 115 quase duplicatas do Weni (Seção 7.1).

As acurácias de validação no split da nuvem foram 99,42% (v2), 99,63% (v3), 99,31% (v4), 99,63% (v5), 99,70% (v6) e 99,49% (v3-pub). Computação: as quatro sessões de fine-tuning usaram 20.476 s, cerca de 5,69 h de tempo de notebook; uma execução do e5-base levou cerca de 31–33 min e uma do e5-large cerca de 2 h 15 min a 2 h 22 min. A v6 e a v3-pub levaram mais 12.713 s e 7.056 s. As três rodadas de sonda gastaram 10.595 s (cerca de 2,94 h) calculando embeddings em CPU. O Apêndice A lista todos os custos.

Tabela 4: Acurácia por frase (%) por rodada. “—” = não rodado. Fumaça informa acertos/10. O número da v3 no Weni está inflado por vazamento (Seção 4d; a v3-pub descontaminada detecta 68,0%, Tabela 5).

Teste (n)	0	A	B	v2	v3	v4	v5	mistura
Weni, pt-BR nativo (300)	—	—	72,3	74,3	94,7	66,7	89,7	91,3
xTRam1, en (1.500)	86,3	89,1	94,3	98,9	98,7	99,0	99,1	99,0
xTRam1-pt, MT (500)	—	81,8	82,8	95,4	96,0	96,0	98,0	96,8
MASSIVE-pt, benigno (1.500)	—	—	99,3	99,7	99,8	99,8	100	99,9
Dolly, benigno en (1.000)	—	—	—	99,7	99,6	99,7	—	—
Dolly-pt, benigno MT (1.000)	—	—	—	99,6	99,3	99,9	99,5	99,5
pt-native (1.800)	—	—	94,8	95,5	98,9	94,3	—	—
deepset (116)	81,9	81,0	84,5	94,0	—	93,1	—	—
jackhhao (262)	95,8	96,6	93,1	99,2	—	98,9	—	—
SPML (1.000)	98,9	98,7	98,8	99,4	—	99,3	—	—
rikka (1.276)	—	—	88,8	95,6	—	94,5	—	—
ShieldLM (4.000)	—	—	96,7	98,7	—	98,8	—	—
yanis (219)	—	—	98,6	100	—	99,1	—	—
HackAPrompt (1.000)	—	—	—	—	—	99,8	—	—
deepset-pt, MT (116)	—	86,2	85,3	94,8	—	92,2	—	—
jackhhao-pt, MT (262)	—	90,1	91,2	98,5	—	97,3	—	—
SPML-pt, MT (300)	—	98,7	98,3	99,3	—	98,7	—	—
HackAPrompt-pt, MT (1.000)	—	—	—	—	—	99,9	—	—
Fumaça (10)	5/10	9/10	9/10	9/10	9/10	9/10	9/10	9/10

4 Resultados por frase

A Tabela 4 relata a acurácia de cada rodada em cada teste em que ela foi rodada. Para o Weni, que contém apenas injeções, a acurácia é igual à taxa de detecção. As fontes são dadas por coluna.

Os números da v3 se restringem aos testes que decidiram o critério; o deepset, o jackhhao e os primeiros 300 itens do SPML foram medidos depois, na comparação da v6 (Tabela 10), e o rikka, o ShieldLM e o yanis nunca. Seguem cinco observações.

(a) O backbone maior moveu o teste ouro; o fine-tuning do modelo base mal o moveu. A sonda sobre embeddings congelados chegou a 72,3% no Weni depois de acrescentar dados nativos em português (a única rodada de sonda medida no Weni). O fine-tuning do e5-base em uma combinação maior de dados quase não mudou isso (74,3%); o fine-tuning do e5-large na *mesma* exportação e receita chegou a 94,7%, um ganho de 20,3 pontos vindo da troca de backbone, com uma única execução por modelo. Os dois modelos foram treinados na mesma exportação, com o mesmo vazamento (d), então a comparação entre eles é justa, embora não tenhamos testado se o modelo maior aproveita mais os vizinhos próximos. Enquanto isso, o xTRam1, que compartilha distribuição com parte dos dados de treino (Seção 3.1), já estava em 98,9% com o e5-base.

(b) O critério foi perdido por um texto, e a diferença não tem significância estatística. A v3 detectou 284 dos 300 itens do Weni; o critério exigia 285. O intervalo de Wilson de 95% (Wilson, 1927) da v3 é 91,5–96,7%, que contém 95%; com $n = 300$, a distinção entre passar e reprovar não é estatisticamente significativa. E (d) mostra que o próprio placar dependia de dados de treino vazados: o quase acerto não era um quase aprovado. Os intervalos das outras rodadas no Weni são 85,7–92,6% (v5), 87,6–94,0% (mistura), 69,1–78,9% (v2) e 61,2–71,8% (v4).

(c) Acrescentar o HackAPrompt com a trava anti-vazamento piorou os dois backbones no Weni: $-7,7$ pontos no e5-base (v4 contra v2) e $-5,0$ no e5-large (v5 contra v3). A nossa contabilidade das exportações sugere por que a trava é a principal diferença entre esses pares. A exportação da v4 é a exportação da v2 mais 4.000 itens do HackAPrompt (3.000 originais e 1.000

traduções) menos 4.008 removidos pela trava. Aplicada só aos dados que não são do HackAPrompt, a mesma trava remove 115 exemplos. Se a trava se comporta de forma idêntica nas duas exportações, cerca de 3.893 dos 4.000 itens do HackAPrompt (97%) foram removidos, de modo que a v4 e a v5 diferem da v2 e da v3 principalmente por (i) a remoção de 115 quase duplicatas do Weni e (ii) cerca de 107 itens do HackAPrompt que sobreviveram. Se essa contabilidade estiver certa, ela também sustenta, indiretamente, a afirmação de que o Weni deriva do HackAPrompt, pois implicaria que quase todo ataque bem-sucedido do HackAPrompt na exportação tem um vizinho no Weni acima de cosseno 0,9. As duas leituras são inferências entre duas exportações e aguardam uma contagem direta por fonte (lacuna L1, Seção 10). Elas também pesam em (d): a v5 perdeu os mesmos 115 vizinhos e ainda detecta 89,7%, enquanto a v3-pub e a v6 ficam abaixo de 70% (Tabela 5); se os cerca de 107 itens do HackAPrompt que sobreviveram forem parentes próximos de textos do Weni, a v5 vaza de outro jeito, o que a mesma contagem por fonte mostraria.

(d) O placar da v3 dependia de dados de treino vazados muito mais do que um corte de cosseno sugeria. A v3 foi treinada antes da trava anti-vazamento (Seção 3.4). Aplicar depois a regra da trava marca 115 dos seus exemplos de treino; a v3 detecta todos os 58 textos do Weni que têm um deles como vizinho e 226 dos outros 242 (93,4%), o que estimava a inflação em cerca de 1,3 ponto. O retreino descontaminado mostra que o placar dependia de muito mais do que esses 115 (Tabela 5). Com a mesma receita e sem as duplicatas dos testes, as traduções delas, as 115 quase duplicatas e as linhas do ShieldLM de origem excluída, a v3-pub detecta 68,0% do Weni: só a v3 detecta 81 textos que a v3-pub erra, e o contrário acontece uma vez (McNemar $p < 10^{-6}$). Nos 242 textos “limpos” ela detecta 63,6%, uma queda de 29,8 pontos, contra 13,8 nos 58 contaminados, e erra mais onde a similaridade com os 115 é menor: todo texto do Weni tem um vizinho acima de cosseno 0,82 entre eles, nove em dez acima de 0,85, e dos 29 textos em 0,85 ou abaixo a v3-pub detecta 2. A perda, portanto, não passa principalmente pelos 115. Ou um corte em 0,9 sobre essas similaridades comprimidas deixa vizinhos vazados entre as outras 1.643 linhas removidas, nenhuma delas acima de 0,9 com algum texto do Weni, ou essas linhas, duplicatas de outros testes e as traduções delas, que para o Weni são exemplos de ataque comuns, carregavam detecção que a v3-pub perdeu junto com elas: ela tem 967 exemplos de ataque a menos que a v3 (de 9.436), 513 deles duplicatas exatas de outros testes. Lemos os 94,7% da v3 como inflados por vazamento, mas a queda de 26,7 pontos limita essa inflação por cima, em vez de medi-la, e um controle “v3 menos as 115”, que testaria os 115 diretamente, não foi rodado. O retreino também move o teste dentro da distribuição como esperado: sem as linhas derivadas do xTRam1, a acurácia no xTRam1 cai de 98,7% para 95,4% ($p < 10^{-6}$), e cai o mesmo nos 1.417 itens sem duplicata no treino (98,6% para 95,3%), de modo que a vantagem da v3 ali vinha da distribuição compartilhada, e não dos itens duplicados. A falha por página não muda: a v3-pub, como a v3, avisa nas 217 páginas benignas em todo limiar (Seção 5.2). Medida nos mesmos itens, uma queda pareada de 27 pontos dificilmente é variância entre execuções, embora a variância entre sementes não tenha sido medida (Seção 10).

(e) A calibração não foi ajustada para os modelos com fine-tuning. No limiar de implantação do hook, 0,9, a v3 detecta 91,0% do Weni e marca 0,13% das frases do MASSIVE-pt e 0,7% das do Dolly-pt; a v5 detecta 86,3% do Weni nesse limiar. Fora o critério perdido por um texto e uma frase do teste de fumaça (Seção 6), toda medida por frase fazia o guard parecer pronto.

Tabela 5: Auditoria de vazamento: v3, a v3-pub descontaminada e a v6 nos mesmos itens. Linhas de frase: acurácia em % no limiar 0,5 (no Weni, detecção; nos conjuntos técnicos, a fração de textos benignos não marcados) com intervalos de Wilson de 95%, e valores p do teste exato de McNemar sobre predições pareadas: “p pub/v3” compara a v3-pub com a v3, “p v6/pub” compara a v6 com a v3-pub (a v6 contra a v3 está na Tabela 10). Conjuntos técnicos: perguntas e respostas do Stack Overflow e docstrings Python, todos benignos. A v3-pub é o conjunto de treino da v3 menos 1.758 exemplos (Seção 3.4); a v6 é o da v3 menos as 115 quase duplicatas do Weni, mais 10.000 textos benignos técnicos (Seção 7). “Limpo” e “contam.” separam o Weni conforme o texto tenha ou não uma das 115 como vizinha acima de cosseno 0,9. Linhas de página: janelas de 640/160 na lista congelada da Seção 7.1; aviso falso (AF) nas 217 páginas benignas, detecção nas 60 páginas com injeção e fração das 2.247 janelas benignas com $P \geq 0,5$.

Teste	n	v3	v3-pub	v6	p pub/v3	p v6/pub
Weni	300	94,7 [91,5–96,7]	68,0 [62,5–73,0]	63,7 [58,1–68,9]	$< 10^{-6}$	0,18
Weni, limpo	242	93,4 [89,5–95,9]	63,6 [57,4–69,4]	60,7 [54,5–66,7]	$< 10^{-6}$	0,48
Weni, contam.	58	100,0	86,2	75,9	—	0,11
xTRam1	1.500	98,7 [97,9–99,1]	95,4 [94,2–96,3]	99,3 [98,8–99,6]	$< 10^{-6}$	$< 10^{-6}$
xTRam1-pt	500	96,0 [93,9–97,4]	94,8 [92,5–96,4]	98,2 [96,6–99,1]	0,263	$< 10^{-4}$
deepset	116	94,8 [89,2–97,6]	94,0 [88,1–97,0]	88,8 [81,8–93,3]	1,000	0,070
MASSIVE-pt	1.500	99,8 [99,4–99,9]	99,9 [99,5–100]	100,0 [99,7–100]	1,000	0,500
Perguntas SO	300	33,3 [28,2–38,8]	41,0 [35,6–46,6]	100,0 [98,7–100]	0,002	$< 10^{-6}$
Respostas SO	300	13,3 [9,9–17,6]	31,0 [26,0–36,4]	100,0 [98,7–100]	$< 10^{-6}$	$< 10^{-6}$
Docstrings	300	62,0 [56,4–67,3]	74,0 [68,8–78,6]	100,0 [98,7–100]	$< 10^{-5}$	$< 10^{-6}$
Pág. AF@0,5	217	100,0 [98,3–100]	100,0 [98,3–100]	34,6 [28,6–41,1]	—	—
Pág. AF@0,999	217	100,0 [98,3–100]	100,0 [98,3–100]	19,4 [14,7–25,1]	—	—
Pág. Det.@0,5	60	100,0 [94,0–100]	100,0 [94,0–100]	90,0 [79,9–95,3]	—	—
Janelas $\geq 0,5$	2.247	99,6	98,6	7,2	—	—

5 Avaliação por página

5.1 Protocolo

A implantação pretendida era um hook que roda depois da ferramenta de web fetch do agente, trunca o conteúdo buscado em 6.000 caracteres, divide-o em janelas sobrepostas e avisa o agente quando a probabilidade máxima entre as janelas chega a 0,9; o hook é só de aviso e fail-open. O avaliador por página chama o mesmo caminho de código do hook, portanto mede o que o agente de fato veria.

Páginas benignas. 217 páginas de documentação técnica: a descrição longa (o README renderizado nos metadados do pacote) de pacotes Python instalados nos ambientes de desenvolvimento do autor, uma por pacote, com mais de 1.500 caracteres, excluindo qualquer uma que contenha a palavra “injection”, truncadas em 6.000 caracteres. As 217 existem no PyPI na versão instalada, e 216 são idênticas byte a byte ao arquivo de core metadata da PEP 658 servido pelo PyPI; a restante é reconstruída a partir da descrição da API JSON, com conferência de hash.

Páginas com injeção. 60 páginas: os mesmos READMEs com uma injeção de teste inserida em uma fronteira de parágrafo determinística, 30 tiradas do Weni e 30 dos positivos do split de teste do xTRam1, amostradas por md5. Nenhum dos dois conjuntos foi carregado como fonte de treino, embora parte do split de treino do xTRam1 tenha chegado ao treino pelo ShieldLM e 83 dos seus 1.500 itens de teste tenham duplicata nos dados de treino da v3 (Seção 3.1).

Decisão. Nota da página = máximo sobre as janelas; limiares 0,5; 0,9; 0,95; 0,99 e 0,999. Rodamos duas configurações de janela: a original, de 1.500 caracteres com 200 de sobreposição, e 640/160, adotada depois que descobrimos que as janelas originais excediam o contexto de 256

Tabela 6: Resultados por página da v3.

Janela (caracteres/sobreposição)	Janelas (benignas / com injeção)	Aviso falso em páginas benignas	Deteção (Weni / xTRam1)
1.500 / 200	895 / 260	100,0% em todo limiar	100% / 100%
640 / 160	2.258 / 662	100,0% em todo limiar	100% / 100%

Tabela 7: Mesmo modelo, duas unidades de avaliação (v3, limiar 0,9).

Unidade	Dados benignos	Aviso falso
frase	MASSIVE-pt (1.500, conversacional)	0,13%
frase	Dolly-pt (1.000, instruções)	0,7%
frase	fumaça: “execute o setup.exe e clique em avançar”	P = 1,0
página	217 READMEs técnicos	100%

tokens do modelo (Seção 8); mesmo assim, 9,2% das janelas de 640 caracteres ainda o excedem (Seção 5.4).

5.2 Resultados

Todas as 217 páginas benignas geraram aviso em todo limiar de 0,5 a 0,999, com qualquer das duas janelas; o intervalo de Wilson de 95% da taxa de aviso falso é 98,3–100%. O menor máximo por página entre as páginas benignas foi 0,9999 com janelas de 1.500 caracteres e 0,9998 com janelas de 640 caracteres (valores gravados com quatro casas decimais), e a mediana foi 1,0. Só em um limiar de 0,9999, fora da grade avaliada, uma página deixaria de avisar (aviso falso de 99,54%). A detecção de 100% é, portanto, não informativa: toda página dispara, com ou sem injeção. A v3-pub descontaminada (Seção 4d) falha do mesmo jeito na lista de páginas congelada: 217 de 217 páginas benignas avisam em todo limiar, e 98,6% das suas janelas benignas pontuam pelo menos 0,5 (Tabela 5). Não é o vazamento que quebra o guard em uso.

O guard foi rejeitado para o hook de web fetch, o hook ficou desligado, e os metadados do pack agora declaram que ele só serve para texto conversacional.

5.3 O que o protocolo por página acrescenta

Um benchmark de frases responde “este texto, sozinho, parece um ataque?”. Um guard implantado responde “alguma parte desta página parece um ataque?”, o que agrega muitas janelas (aqui, cerca de 10,4 janelas benignas por página com janelas de 640 caracteres). Com um máximo sobre k janelas, uma taxa de falso positivo por janela p vira uma taxa de aviso falso por página de $1 - (1 - p)^k$ sob independência, de modo que mesmo uma taxa modesta por janela é amplificada cerca de k vezes. No nosso caso a amplificação é irrelevante, porque as janelas benignas já pontuam perto de 1,0; mas a aritmética explica por que taxas de falso positivo por frase abaixo de 1% não tranquilizam para uso por página. As taxas de falso positivo por janela não foram gravadas na execução original (lacuna L4); a execução dos baselines da Seção 5.4 as registrou. Nela, 99,6% das janelas benignas de 640 caracteres da v3 pontuam pelo menos 0,5, contra 3,5–4,3% nos classificadores da ProtectAI e da Proventra, cujos avisos falsos por página são essa amplificação em ação, e 0,5% no Prompt Guard 2 (86M).

Tabela 8: Classificadores públicos no protocolo de página: aviso falso (AF) nas 217 páginas benignas e detecção (Det.) nas 60 páginas com injeção (30 do Weni, 30 do xTRam1), em %. “Janela” é o limite de tokens do modelo e o comprimento da janela em caracteres; as quatro últimas linhas são o controle com 640/160. Modelos: e5-large v3 (o nosso), ProtectAI deberta-v3-base-prompt-injection-v2, Proventra mdeberta-v3-base-prompt-injection, deepset deberta-v3-base-injection, PG2 86M e 22M = Llama-Prompt-Guard-2-86M e -22M. “Det.@0,9” é a detecção a 0,9, no total e (entre parênteses) nas inserções do Weni / xTRam1. O “AUROC” é calculado sobre páginas. “Det.@2%” é a detecção no menor limiar que deixa no máximo 4 das 217 páginas benignas acima dele ($AF \leq 2\%$). “Jan.” é a fração das janelas benignas com $P \geq 0,5$ (não registrada no controle).

Modelo	Janela	AF@0,5	AF@0,9	AF@0,99	Det.@0,9	AUROC	Det.@2%	Jan.
v3 (nosso)	256/640	100,0	100,0	100,0	100 (100/100)	0,857	40	99,6
ProtectAI	512/1.300	15,7	11,1	5,5	58 (97/20)	0,813	50	4,3
Proventra	512/1.300	14,7	12,4	8,3	68 (70/67)	0,883	40	3,5
deepset	512/1.300	100,0	100,0	100,0	100 (100/100)	0,829	32	100,0
PG2 86M	512/1.300	1,8	0,9	0,0	57 (90/23)	0,956	63	0,5
PG2 22M	512/1.300	0,0	0,0	0,0	7 (13/0)	0,639	23	0,0
ProtectAI	512/640	30,9	21,2	8,8	77 (97/57)	0,847	55	—
Proventra	512/640	38,2	33,6	26,7	85 (90/80)	0,845	38	—
PG2 86M	512/640	2,3	0,9	0,5	58 (93/23)	0,923	65	—
PG2 22M	512/640	0,9	0,0	0,0	18 (27/10)	0,616	22	—

5.4 Baselines públicos sob o mesmo protocolo

Rodamos cinco classificadores públicos sob o mesmo protocolo de página: dois modelos DeBERTa-v3 em inglês (deepset, 2023a; ProtectAI.com, 2024), um modelo mDeBERTa multilíngue (Proventra, 2025) e os dois tamanhos do Llama Prompt Guard 2 (Meta, 2025), o de 86M sobre o mesmo backbone mDeBERTa-v3 e o de 22M sobre o DeBERTa-v3-xsmall, em inglês. Cada um lê janelas dimensionadas para o próprio contexto (1.300/200 caracteres com 512 tokens; 640/160 com 256 tokens para o nosso), e um controle com janelas de 640/160 foi rodado para todo modelo público, exceto o da deepset, cuja saída já satura com janelas de 1.300 caracteres. O Prompt Guard 2 foi medido em 7 de outubro, um dia depois dos outros, quando o acesso ao seu repositório restrito (gated), que exige aceitar a Llama 4 Community License, foi concedido; o seu antecessor (Meta, 2024), cujo acesso é pedido à parte, continuou respondendo HTTP 403 e não foi avaliado. A configuração do Prompt Guard 2 só chama as suas duas classes de LABEL_0 e LABEL_1, então antes de cada execução conferimos que LABEL_1 é a classe maliciosa numa injeção conhecida e numa frase benigna. O modelo da Proventra, escolhido como representante multilíngue enquanto o Prompt Guard estava inacessível, continua na comparação; o seu card relata injeções embutidas em sites e artigos entre os seus dados de treino. Os seis modelos leram as mesmas páginas exportadas, e cada página recebe o máximo, sobre as suas janelas, do log-odds do rótulo de ataque, com os limiares aplicados à probabilidade correspondente; o log-odds mantém uma ordenação onde as probabilidades arredondam para 1. Os números por frase usam os 300 itens do Weni e o split de teste completo do xTRam1 em inglês (2.060 frases, 650 ataques), com o mesmo hash dos textos em todas as execuções. O nosso modelo e os três primeiros modelos públicos rodaram na mesma VM ARM de nuvem com 4 núcleos, em CPU e fp32; o Prompt Guard 2 rodou com o mesmo script, a mesma versão da biblioteca e as mesmas entradas em instâncias de CPU x86 de um serviço de notebooks na nuvem, o que mantém comparáveis os números de qualidade, mas não a latência (Tabela 9). As Tabelas 8 e 9 trazem os resultados.

A falha não é exclusiva do nosso modelo, nem universal. O classificador da deepset também avisou nas 217 páginas benignas em todo limiar até 0,99 (IC de Wilson de 95%: 98,3–100%)

Tabela 9: Os mesmos classificadores em frases, no limiar 0,5, e o custo por janela cheia na VM ARM de 4 núcleos (uma janela por chamada). O nosso número no Weni está inflado por vazamento (Seção 4d); a v3-pub descontaminada detecta 68,0%. Weni: detecção nas 300 injeções nativas em pt-BR. xTRam1: split de teste completo em inglês, 2.060 frases (650 ataques); TPR e FPR em %. $\approx$: o Prompt Guard 2 foi cronometrado em instâncias x86 de notebook (1.138 e 433 ms por janela nelas); o custo na ARM é o tempo da instância multiplicado pela razão do modelo da Proventra cronometrado na mesma instância, uma estimativa, e não uma medida.

Modelo	Weni det.	xTRam1 acur.	TPR	FPR	AUROC	ms/janela
e5-large v3 (nosso)	94,7	98,7	98,9	1,3	0,999	1.045
ProtectAI v2	99,3	94,9	84,2	0,1	0,992	868
Proventra mDeBERTa	79,0	89,6	72,6	2,6	0,906	845
deepset	100,0	48,3	98,9	75,0	0,666	879
Prompt Guard 2 86M	93,7	84,7	51,7	0,1	0,973	$\approx$ 847
Prompt Guard 2 22M	16,3	77,3	28,2	0,0	0,893	$\approx$ 305

e, em frases, marca 75,0% dos itens benignos do xTRam1. O ProtectAI v2 e o modelo mDeBERTa da Proventra avisaram em 15,7% (34/217; IC 11,4–21,1%) e 14,7% (32/217; IC 10,6–20,1%) das páginas benignas no limiar 0,5, e em 5,5% e 8,3% no limiar 0,99. O Prompt Guard 2 avisou em muito menos: o de 86M em 1,8% (4/217; IC 0,7–4,6%) no limiar 0,5 e em nenhuma no limiar 0,99, e o de 22M em nenhuma, em qualquer limiar (IC 0–1,7% a 0,5). No ProtectAI e no Proventra a janela benigna mediana pontua 0,00028 e 0,00123, contra 0,99998 no nosso, e o aviso falso por página nasce de poucas janelas (3,5–4,3% das janelas benignas pontuam pelo menos 0,5) somadas ao longo de cerca de 4,9 janelas por página: a amplificação da Seção 5.3. No Prompt Guard 2 (86M) a janela benigna mediana pontua 0,004, e só 0,5% das janelas benignas chegam a 0,5. Esses cards relatam dados de treino mais variados que os nossos: o da ProtectAI lista mais de vinte fontes, incluindo instruções e conversas; o da Proventra relata injeções embutidas em conteúdo legítimo, como sites e artigos; e o do Prompt Guard 2 relata dados benignos da web e um termo de perda baseado em energia, aplicado aos prompts benignos, ao qual atribui melhor precisão em dados fora da distribuição.

Um modelo público passa no critério de página com detecção útil, por pouco. No limiar 0,5, dois modelos ficam abaixo de 2% de aviso falso por página, ambos Prompt Guard 2, e só o de 86M detecta uma parte útil das páginas com injeção: ele avisa em 4 de 217 páginas benignas (1,8%; IC de 95% de 0,7–4,6%, de modo que 217 páginas não bastam para mostrar que ele fica abaixo de 2% com folga) e detecta 62% (37 de 60; 49–73%). No limiar 0,9 ele avisa em 0,9% e detecta 57%, e o seu AUROC por página, 0,956, é o maior da comparação. A detecção, porém, vem quase toda das inserções longas do Weni (93%, contra 30% do xTRam1), e em frases ele não atinge o critério do Weni (93,7% contra 95%) e detecta 51,7% dos ataques do xTRam1 em inglês com 0,1% de falsos positivos. Parte da diferença no xTRam1 é de definição: o card só conta como malicioso a tentativa explícita de passar por cima de instruções anteriores, e os ataques do xTRam1 que ele deixa passar incluem pedidos de prontuário ou de credencial e o pedido de um poema com um primeiro verso prescrito, sem nenhuma instrução desse tipo; a maioria das 19 frases do Weni que ele perde são ofuscações no estilo da HackAPrompt. O de 22M, cujo card avisa que o backbone não tem pré-treino multilíngue, não avisa em nenhuma página (0 de 217; IC 0–1,7%), e por isso passa no critério de página com folga, mas detecta só 15% das páginas com injeção (9 de 60; 8–26%; Weni 20%) e 16,3% das frases do Weni. Nos demais modelos, a detecção no menor limiar que mantém o aviso falso em até 2% fica entre 23% e 50% no protocolo principal e chega no máximo a 55% no controle. O único ponto deles na grade avaliada abaixo de 2% de aviso falso com detecção não trivial é o ProtectAI no limiar 0,999 (0,9% de aviso falso, 50% de detecção), e quase

toda essa detecção também vem das injeções longas do Weni (90%, contra 10% do xTRam1). Com 60 páginas com injeção, o intervalo de 95% da detecção cobre cerca de ± 12 pontos: o ProtectAI detecta 38 de 60 páginas a 0,5 (51–74%) e o Proventra, 42 de 60 (57–80%).

Frase e página ordenam os modelos em sentidos opostos. Nas frases do xTRam1, o nosso modelo tem a maior acurácia (98,7%) e o maior AUROC (0,999) dos seis, num teste que está dentro da distribuição dele (Seção 3.1; a v3-pub faz 95,4% no recorte de 1.500 itens da Tabela 5); por página, empata com o da deepset no maior aviso falso. O Prompt Guard 2 (86M) mostra a inversão pelo outro lado: terceiro no AUROC por frase do xTRam1 (0,973, atrás do nosso e do ProtectAI), tem o melhor AUROC por página (0,956). O benchmark de frases não previu o comportamento na unidade em que o hook decide.

Saturação, não limiar. 98,2% das nossas janelas benignas pontuam pelo menos 0,999. No modelo da deepset a saída é quase constante: o log-odds da página vai de 6,41 a 6,79 nas páginas benignas e de 6,65 a 6,88 nas páginas com injeção (percentis 5 a 95), e por isso o limiar 0,999 derruba de uma vez o aviso falso e a detecção (0% e 0%). O nosso ainda guarda alguma ordem (AUROC por página de 0,857), mas o limiar que leva o aviso falso a 2% fica em $P > 1 - 5 \times 10^{-6}$, que depende da quinta casa decimal de uma saída saturada e não serve como ponto de operação. O Prompt Guard 2 (86M) é o caso oposto: o log-odds da página benigna tem mediana $-4,9$ (percentil 95 em $-3,2$), contra $+3,6$ na página com injeção, e o seu ponto de operação de 2% fica em $P > 0,18$, no meio da faixa de saída. Já a saída do de 22M fica comprimida em janelas longas (probabilidade da página entre cerca de 0,2 e 0,36 para quase toda página), de modo que ele nem avisa nem separa (AUROC por página de 0,639), embora separe melhor frases curtas isoladas (AUROC de 0,893 no xTRam1). Isso é coerente com um defeito de dado de treino, e não de limiar; o retreino v6 (Seção 7) testa isso diretamente.

O tamanho da janela é um parâmetro de operação. Com a janela pela metade (640/160), o aviso falso a 0,5 praticamente dobrou (ProtectAI 15,7% $\rightarrow$ 30,9%, Proventra 14,7% $\rightarrow$ 38,2%) e a detecção subiu (63% $\rightarrow$ 80% e 70% $\rightarrow$ 88%): as injeções curtas do xTRam1 (mediana de 136 caracteres) se diluem menos numa janela pequena (detecção por página do ProtectAI no xTRam1: 30% $\rightarrow$ 63%), enquanto cada página passa a ter cerca do dobro de janelas para errar. O Prompt Guard 2 (86M) quase não se mexeu (aviso falso de 1,8% $\rightarrow$ 2,3%, detecção de 62% $\rightarrow$ 65%, AUROC por página de 0,956 $\rightarrow$ 0,923). Mesmo com 640 caracteres, 9,2% das janelas do nosso modelo ainda passam do contexto de 256 tokens, contra 6,7% (ProtectAI, deepset, Prompt Guard 2 22M) e 1,9% (Proventra e Prompt Guard 2 86M, que compartilham o tokenizador do mDeBERTa) das janelas de 1.300 caracteres com 512 tokens.

Custo. Na VM ARM, uma janela cheia leva cerca de 1,05 s no nosso e5-large (560M parâmetros) e 0,85–0,88 s nos modelos DeBERTa-base (184M); juntar em lote não ajuda nesta CPU. Uma página média (4.736 caracteres) custa, portanto, cerca de 11 s no nosso modelo (10,4 janelas) e cerca de 4 s nos DeBERTa com 1.300 caracteres (4,9 janelas), um atraso relevante para um hook síncrono de busca na web. O Prompt Guard 2 foi cronometrado nas instâncias de notebook ao lado do modelo da Proventra, como âncora: lá o de 86M custa o mesmo que o da Proventra (1.138 contra 1.136 ms por janela; mesmo backbone) e o de 22M, 36% dele, o que os põe em cerca de 4 s e 1,5 s por página média na VM ARM, por conversão linear.

Ressalvas. As páginas benignas são de um único gênero, e as injeções são texto colado, não marcação escondida. Os dados de treino dos modelos públicos não estão documentados por inteiro; se o Weni deriva da HackAPrompt (Seção 3.4), um modelo treinado com a HackAPrompt pode ter o Weni inflado, e o card do Prompt Guard 2 também não lista as suas bases de treino. A definição mais estreita de ataque do Prompt Guard 2 baixa a sua detecção nos dois conjuntos de teste e pode também ajudar a manter limpas as suas páginas benignas. Os números do nosso modelo no xTRam1 carregam a ressalva de sobreposição da Seção 3.1. Tampouco o Weni é limpo para

nós: os nossos 94,7% estão inflados por vazamento (Seção 4d), e, diante da v3-pub descontaminada (68,0%), os modelos públicos com maior detecção no Weni, o ProtectAI v2 (99,3%) e o Prompt Guard 2 86M (93,7%), ficam muito à frente, com a ressalva que acabamos de fazer sobre os seus dados de treino. O script dos baselines reproduz os 94,7% de detecção no Weni da Seção 4, o que confere que ele usa o mesmo caminho de inferência do kit implantado.

6 Diagnóstico

6.1 Hipótese: “texto técnico imperativo = injeção”

A nossa explicação de trabalho, registrada no dia da falha, é que o modelo aprendeu a tratar texto técnico e imperativo como injeção porque os exemplos benignos de treino eram conversacionais (comandos a assistentes do MASSIVE e instruções do Dolly). Corpora de injeção estão cheios de imperativos dirigidos a um sistema (“ignore”, “imprima”, “execute”, “revele”); os corpora benignos que usamos também contêm imperativos, mas conversacionais, dirigidos a um assistente. A documentação técnica fica do lado da injeção dessa fronteira: imperativos na segunda pessoa, comandos de shell, diretivas de configuração, avisos sobre o que sobrescrever. **Essa explicação era uma hipótese quando a falha foi encontrada.** Ela é consistente com as evidências abaixo; o retreino v6 (Seção 7) a testa, e a sustenta para a prosa técnica, deixando avisos falsos residuais na marcação dos READMEs; e a v3-pub descontaminada, que continua avisando em toda página, descarta o vazamento como causa (Seção 7.2).

6.2 Evidências

O sinal precoce estava no teste de fumaça. A frase benigna de fumaça “Instruções de instalação: execute o setup.exe e clique em avançar.” foi classificada como injeção em toda rodada a partir da A em que a nota por item foi gravada (A, B, v3, v5), com confiança crescente: $P = 0,883$ na rodada A, $0,967$ na rodada B e $1,0$ na v3; na v2, na v4 e na mistura, uma das cinco frases benignas de fumaça foi classificada errado, mas qual delas não foi gravado. Na rodada 0 a sonda errou todas as cinco injeções em português (maior $P = 0,186$) e classificou corretamente as cinco frases benignas, então a frase ainda não estava em questão. Uma frase técnica entre cinco benignas era o único item escrito para representar a distribuição de implantação em toda a suíte de avaliação, e o seu erro foi lido como um erro isolado, e não como uma classe de erros.

A documentação estava ausente do treino. Uma auditoria do conjunto de treino da v6, que contém literalmente 23.578 dos 23.693 textos de treino da v3, contra os 329 textos de metadados de pacotes da máquina não encontrou nenhum texto de treino idêntico a um README, nenhum texto de treino contido em um README (≥ 200 caracteres) e nenhum README contido no conjunto de treino; 9 textos de treino compartilhavam um 13-grama com algum README, todos boilerplate XML/XHTML. A mesma auditoria da v3-pub, um subconjunto estrito dos dados de treino da v3, contra 334 textos de metadados não achou nada em comum, nem um 13-grama. Nenhum dos READMEs do benchmark, nem qualquer fragmento longo deles, estava entre os exemplos benignos de treino; não medimos quanto texto parecido com README, vindo de outras fontes, os dados de treino continham.

A saturação elimina o limiar como controle. Como os modelos com fine-tuning foram servidos sem calibração de temperatura, suas saídas saturam: a frase de fumaça e quase toda página recebem $P = 1,0$ com quatro casas decimais. O placar mínimo de página de $0,9998$ não deixa limiar utilizável: mesmo em $0,9999$ só uma das 217 páginas benignas deixaria de avisar. A calibração em validação dentro da distribuição não repararia isso sozinha: o temperature scaling é monotônico,

então pode mover o limiar, mas não mudar a ordem em que as páginas são ranqueadas; o split de validação compartilha a distribuição benigna conversacional; e as rodadas de sonda, que foram calibradas ($T = 1,1286$ e $1,2346$), ainda deram à frase do setup.exe nota de 0,883–0,967.

A falha sobrevive a uma janela correta. Com janelas de 1.500 caracteres o classificador via apenas os primeiros 256 tokens de cada janela, já que READMEs técnicos têm uma mediana registrada de cerca de 3,04 caracteres por token (Seção 8; o script de medição não foi preservado). Reduzir a janela para 640 caracteres, de modo que a maioria das janelas caiba no contexto (9,2% ainda passam de 256 tokens; Seção 5.4), não mudou o resultado: 100% de avisos falsos em todo limiar nas duas configurações. O truncamento era um bug real, mas não é a causa.

6.3 O que o modelo vê

Cada janela entra no classificador como `query:␣` + até 256 tokens de texto de README. Uma janela da página de um pacote típico contém um comando de instalação, um trecho de uso e instruções na segunda pessoa (“defina esta variável”, “rode este comando”, “não use X em produção”). Do ponto de vista do classificador, essas são strings imperativas curtas dirigidas a um agente executor, que é a cara da maioria das injeções de treino e a cara de poucos exemplos benignos de treino. O lado benigno dos dados de treino lhe ensinou como é uma *conversa* inofensiva, não como é um *conteúdo* inofensivo. As páginas benignas de nota mais alta na execução dos baselines (Seção 5.4) combinam com essa leitura. No nosso modelo, são blocos de instalação e de teste, avisos de depreciação e tabelas com emoji, ou seja, texto técnico no imperativo; os modelos públicos erram em outros lugares: o ProtectAI em código que pede um segredo ou uma senha ao usuário e num link para uma política de segurança, o modelo mDeBERTa em código que lê variáveis de ambiente e numa tabela de técnicas de ataque, o da deepset em texto de licença e de contribuição, e o Prompt Guard 2 (86M), nas suas quatro páginas benignas acima de 0,5, em trechos fora da prosa: uma tabela de marcas de verificação, badges e um cabeçalho, um exemplo de terminal com mensagem de erro e uma tabela de opções. Depois do retreino (Seção 7), as janelas que ainda disparam são de outro tipo: badges, links de patrocínio e tabelas markdown.

7 Tentativa de correção: dados benignos técnicos (v6)

7.1 Desenho

A rodada v6 testa diretamente a hipótese da Seção 6 acrescentando texto benigno técnico ao treino: perguntas e respostas do Stack Overflow (koutch, 2023; pacovaldez, 2022) e docstrings Python do CodeSearchNet (Husain et al., 2019; Nan-Do, 2023), 10.000 exemplos de treino no total, com 1.000 exemplos de teste por fonte. Metadados de pacotes **nunca** são usados no treino, porque são o teste de página. O conjunto de treino da v6 é o conjunto de treino da v3 menos as 115 quase duplicatas do Weni encontradas pela trava anti-vazamento, mais os 10.000 exemplos benignos técnicos, num total de 33.578 (24.240 benignos, 9.338 injeções); o HackAPrompt não entra. A fração de positivos cai de 39,8% (v3) para 27,8% (v6). A receita é, no mais, idêntica à da v3 (e5-large, 3 épocas, taxa de aprendizado $2e-5$, batch 8×4 , fp16, 256 tokens, temperatura 1,0 na inferência).

A v6 é comparada com a v3 nos mesmos itens: intervalos de Wilson de 95% por teste e testes exatos de McNemar (McNemar, 1947) sobre predições pareadas, já que os dois modelos veem exatamente os mesmos textos, na mesma ordem. A avaliação repete a da v3: os mesmos testes, textos, código e máquina ARM. Os testes em que a v3 não tinha sido rodada (deepset, jackhhao, SPML e os três conjuntos técnicos) foram rodados para os dois modelos nos mesmos 300 primeiros itens. As páginas são a lista congelada usada na Seção 5.4, com a mesma impressão digital para

Tabela 10: v3, v3-pub e v6 em frases no limiar 0,5: acurácia em % (no Weni, detecção), nos mesmos itens para os três modelos, com o teste exato de McNemar da v6 contra a v3 sobre predições pareadas (os testes pareados da v3-pub estão na Tabela 5 e no texto). Os números da v3 no Weni estão inflados por vazamento (Seção 4d). Alguns n diferem da Tabela 4: os testes deepset, jackhhao, SPML, Dolly e técnicos usam os seus primeiros 300 itens (todos, quando há menos). “Weni, limpo” exclui os 58 textos com uma quase duplicata nos dados de treino da v3. Os testes técnicos são separados por hash das novas fontes de treino da v6.

Teste	n	v3	v3-pub	v6	v6 – v3 (p.p.)	McNemar p, v6/v3
Weni, pt-BR nativo	300	94,7	68,0	63,7	−31,0	$< 10^{-6}$
Weni, limpo	242	93,4	63,6	60,7	−32,6	$< 10^{-6}$
xTRam1, en	1.500	98,7	95,4	99,3	+0,7	0,031
xTRam1-pt, MT	500	96,0	94,8	98,2	+2,2	0,007
MASSIVE-pt, benigno	1.500	99,8	99,9	100,0	+0,2	0,250
Dolly, benigno en	300	99,7	99,7	99,3	−0,3	1,000
Dolly-pt, benigno MT	1.000	99,3	99,8	99,6	+0,3	0,250
deepset, en	116	94,8	94,0	88,8	−6,0	0,016
deepset-pt, MT	116	93,1	92,2	89,7	−3,4	0,125
jackhhao, en	262	98,9	99,2	99,2	+0,4	1,000
jackhhao-pt, MT	262	98,1	98,9	98,5	+0,4	1,000
SPML, en	300	100,0	100,0	100,0	0,0	1,000
SPML-pt, MT	300	100,0	100,0	100,0	0,0	1,000
Perguntas do SO, benigno técnico	300	33,3	41,0	100,0	+66,7	$< 10^{-6}$
Respostas do SO, benigno técnico	300	13,3	31,0	100,0	+86,7	$< 10^{-6}$
Docstrings, benigno técnico	300	62,0	74,0	100,0	+38,0	$< 10^{-6}$
Fumaça (acertos/10)	10	9/10	9/10	10/10	—	1,000

os dois modelos: entre as duas execuções, pacotes novos nos ambientes do autor tinham trocado 21 das 217 páginas selecionadas ao vivo e alterado outras 2, o que invalidaria a comparação. As probabilidades foram gravadas com quatro casas decimais, o que limita a análise de limiar perto de 0 e de 1. A aceitação exige taxa de aviso falso por página abaixo de 2%, com a detecção relatada.

Um controle descontaminado. A v6 mudou três coisas ao mesmo tempo: acrescentou negativos técnicos, tirou as 115 quase duplicatas do Weni e baixou a fração de positivos. A v3-pub (Seção 3.4), treinada um dia depois, é um controle parcial. Como a v6, ela não tem as 115 quase duplicatas do Weni, e, ao contrário da v6, não tem negativos técnicos; mas ela também tira as duplicatas dos testes, as traduções delas e as linhas do ShieldLM de origem excluída que a v6 mantém (1.758 exemplos removidos contra 115), e a sua fração de positivos é de 38,6%, contra 27,8% na v6. Comparar a v6 com a v3-pub isola, portanto, os negativos técnicos melhor do que comparar a v6 com a v3, mas não é uma ablação. A v3-pub foi avaliada nos mesmos itens, páginas congeladas, código e máquina da v3 e da v6, e pareada contra as duas.

7.2 Resultados

A Tabela 10 compara v3, v3-pub e v6 em frases e a Tabela 11, em páginas; a Tabela 5 traz os intervalos e os testes pareados das linhas principais.

O diagnóstico se sustenta para a prosa técnica. Em frases técnicas benignas separadas do treino, a v3 marcava 66,7% das perguntas do Stack Overflow, 86,7% das respostas e 38,0% das docstrings; a v6 não marca nenhuma das 900. A descontaminação sozinha ajuda bem menos: a v3-pub, sem negativos técnicos, ainda marca 59,0%, 69,0% e 26,0% delas (51,3% no conjunto, contra 63,8% na v3). Esses testes vêm das mesmas fontes dos novos dados de treino da v6, portanto estão dentro da distribuição dela; o teste de página não está, já que nenhum README entrou no

Tabela 11: v3 contra v6 em páginas (janelas de 640/160, lista de páginas congelada): aviso falso (AF) nas 217 páginas benignas e detecção (Det.) nas 60 páginas com injeção, em %, com intervalos de Wilson de 95% para a v6. A v3 avisa em toda página benigna e detecta toda página com injeção em todo limiar (intervalos 98,3–100% e 94,0–100%), e a v3-pub também (Tabela 5).

Limiar	AF v3	AF v6 [IC 95%]	Det. v3	Det. v6 [IC 95%]	Det. v6, Weni / xTRam1
0,5	100,0	34,6 [28,6–41,1]	100,0	90,0 [79,9–95,3]	90,0 / 90,0
0,9	100,0	30,4 [24,7–36,8]	100,0	85,0 [73,9–91,9]	86,7 / 83,3
0,95	100,0	29,0 [23,4–35,4]	100,0	85,0 [73,9–91,9]	86,7 / 83,3
0,99	100,0	24,0 [18,8–30,1]	100,0	83,3 [72,0–90,7]	86,7 / 80,0
0,999	100,0	19,4 [14,7–25,1]	100,0	81,7 [70,1–89,4]	83,3 / 80,0

treino (Seção 6.2). Nos READMEs, a fração de janelas benignas de 640 caracteres com nota de pelo menos 0,5 caiu de 99,6% (v3) e 98,6% (v3-pub) para 7,2% (v6), uma redução de catorze vezes.

A correção não basta para o hook. Os avisos falsos por página caíram de 100% para 34,6% no limiar 0,5 e para 19,4% em 0,999, e a detecção nas páginas com injeção passou a ser informativa (90,0% e 81,7%). Como a v3-pub, descontaminada mas sem negativos técnicos, continua avisando em toda página benigna, essa queda se deve aos negativos técnicos, e não à remoção dos exemplos vazados. O critério de página continua fora de alcance: 26 das 217 páginas benignas (12,0%) ainda pontuam 1,0000 na resolução gravada, de modo que nenhum limiar leva o aviso falso a 2%. O AUROC por página é 0,868 na v6; calculado do mesmo modo, sobre probabilidades com quatro casas, a v3 fica com 0,505 (a v3-pub, com 0,500) porque quase todas as páginas delas empatam em 1,0, enquanto os seus log-odds na execução dos baselines dão 0,857 (Tabela 8). A qualidade da ordenação é, portanto, parecida; o que mudou é que a ordenação agora fica numa faixa utilizável da saída. Diante dos baselines públicos, a v6 avisa em mais páginas benignas no limiar 0,5 que o ProtectAI v2 e o modelo mDeBERTa (34,6% contra 15,7% e 14,7%) e detecta mais páginas com injeção (90% contra 63% e 70%): ela ocupa outro ponto da mesma troca, e não um ponto além dela. Com as mesmas janelas de 640/160, o Prompt Guard 2 (86M) ordena as páginas melhor que a v6 (AUROC de 0,923 contra 0,868, este calculado sobre probabilidades com quatro casas) e opera com 2,3% de aviso falso e 65% de detecção no limiar 0,5. As janelas que ainda disparam, uma mediana de 2 por página marcada, são boilerplate de README que não existe no Stack Overflow nem em docstrings: badges em HTML, links de patrocínio e de doação, tabelas markdown e listas de links.

A queda no Weni não é claramente da correção. No Weni, a v6 detecta 63,7% contra 94,7% da v3 (94 textos contra 1; McNemar $p < 10^{-6}$), e 60,7% contra 93,4% nos 242 textos limpos. Lemos isso primeiro como o preço de tirar o atalho. O controle descontaminado enfraquece essa leitura: a v3-pub, que não tem negativos técnicos, detecta 68,0% (63,6% nos textos limpos). Contra ela, a v6 perde 4,3 pontos, com 47 textos detectados só pela v3-pub e 34 só pela v6 (McNemar $p = 0,18$; nos textos limpos, 39 contra 32, $p = 0,48$). Uma leitura é que o que a v6 e a v3-pub têm em comum, a perda dos 115 exemplos de treino mais próximos do Weni, responde pela maior parte das duas quedas, e os negativos técnicos, por pouco no limiar 0,5. Os totais se encaixam nela; o padrão dos erros, menos. A v5 perdeu os mesmos 115 vizinhos e manteve 89,7%, embora também tenha ganhado cerca de 107 ataques do HackAPrompt que podem ser parentes do Weni (Seção 4c). Nenhum dos dois modelos perde mais onde a similaridade com os 115 é maior (acima de 0,9, a v3-pub detecta 50 de 58 textos e a v6, 44), e os dois se separam onde ela é menor: dos 29 textos do Weni menos parecidos com os 115, a v6 detecta 20 e a v3-pub, 2. E os dois modelos erram textos em parte diferentes: 62 são errados pelos dois, 47 só pela v6 e 34 só pela v3-pub. A perda da v3-pub pode, então, passar pelos seus cortes mais amplos (Seção 4d), que a v6 não tem, e nesse caso as duas quedas têm causas diferentes e a da v6 deve mais aos negativos novos do que os 4,3 pontos

sugerem. As bases também diferem (Seção 7.1). O controle “v3 menos as 115” decidiria entre essas leituras. Na ordenação, o custo dos negativos novos é mais claro. Pareada com negativos do tipo assistente, a v3-pub ainda separa bem o Weni (AUC 0,977; 95,7% de detecção com taxa de falso positivo de 2%), de modo que os seus erros em 0,5 são sobretudo de calibração; a v6 o separa pior (AUC 0,883; 77,0%) e dá $P = 0,0000$ a 23% dos textos do Weni, contra 4,3% na v3-pub, de modo que baixar o limiar quase não os recupera (67,0% em 0,1 e 69,3% em 0,01, contra 73,0% e 78,0% na v3-pub). Pareadas com os negativos técnicos, a ordem se inverte: a v3 e a v3-pub não detectam nenhuma injeção do Weni com taxa de falso positivo de 2% (AUC 0,843 e 0,655, na resolução gravada), enquanto a v6 detecta 77% (AUC 0,885). Os textos que a v6 perde em relação à v3 são curtos (705 caracteres em média, contra 1.101 dos demais) e apresentados como tarefas legítimas (“gere um título de filme sobre a frase...”, “traduza...”, “você é um mecanismo de busca...”), com a frase-alvo do ataque fatiada ou ofuscada. Os outros testes com ataques ficam iguais ou melhoram (xTRam1-pt 96,0% $\rightarrow$ 98,2%, $p = 0,007$; jackhhao e SPML sem mudança), exceto o deepset em inglês (94,8% $\rightarrow$ 88,8%, $p = 0,016$), em que a v3-pub fica em 94,0% (v6 contra v3-pub: $p = 0,07$).

Leitura. Dos dois efeitos que a nossa primeira leitura confundia, um agora se separa com firmeza; o outro continua em aberto. Com firmeza: são os negativos técnicos, e não a descontaminação, que eliminam os avisos falsos em texto técnico e nas páginas; eles levam os avisos falsos técnicos de 26–69% na v3-pub a 0% e os avisos falsos por página de 100% a 34,6% no limiar 0,5, enquanto a descontaminação sozinha baixa os avisos falsos técnicos só em parte (de 63,8% para 51,3% no conjunto) e não muda nada nas páginas. Em aberto: o que causa a maior parte da queda da v6 no Weni. Um modelo descontaminado sem negativos técnicos cai quase o mesmo em 0,5, então a comparação com a v3 não mostra que os negativos a causaram; o padrão dos erros e a v5 pesam contra uma causa comum, e na ordenação contra negativos conversacionais o custo dos negativos aparece. O controle que isola os 115 vizinhos do Weni não foi rodado. Nenhuma das três versões atende aos dois critérios: a v3 erra o Weni por um texto com um placar inflado e reprova no critério de página, e a v3-pub e a v6 reprovam nos dois. A v3 continua no kit apenas para texto conversacional, e o hook continua desligado.

8 Lições de engenharia para a integração com agentes

O guard foi testado de ponta a ponta antes de ser ligado, e esse teste encontrou três defeitos no hook (I1–I3) antes mesmo que a falha por página ficasse visível; um defeito relacionado, de formato de resposta, no cliente do classificador (I4) tinha sido corrigido mais cedo no mesmo dia. Listamos os defeitos que se generalizam para qualquer classificador colocado dentro do laço de um agente (Tabela 12; o registro completo de incidentes está no Apêndice B).

Lição 1: erros de encoding são entradas com cara de ataque. O I1 é o mais instrutivo. Um guard é treinado para disparar com texto estranho, fora de lugar; mojibake é texto estranho, fora de lugar. Um bug de encoding no código de cola, portanto, não só degrada a acurácia: ele fabrica falsos positivos que parecem o guard fazendo o seu trabalho.

Lição 2: fail-open precisa de sinal de vida. Fail-open é o padrão certo para um guard só de aviso: um guard quebrado não pode quebrar o agente. Mas o I3 mostra o custo: uma incompatibilidade de versão entre cliente e servidor transformou cada chamada em exceção, cada exceção em “sem aviso”, e o hook em um no-op silencioso com aparência saudável. Um componente fail-open precisa relatar que falhou, por exemplo com um contador ou com um autoteste periódico sobre um positivo conhecido; do contrário, “nenhum aviso” é indistinguível de “tudo limpo”.

Lição 3: fixe e teste o contrato, não só o modelo. I3 e I4 são bugs de contrato entre um cliente e um servidor de decisão em versões diferentes. Um teste de contrato que mande uma

Tabela 12: Defeitos de integração.

#	Sintoma	Causa raiz	Correção
I1	O hook alimentava o modelo com mojibake (“mÃ³dulo”)	No Windows, o Python decodifica o stdin como cp1252; o agente envia UTF-8. Mojibake parece injeção para o classificador	ler <code>sys.stdin.buffer</code> e decodificar UTF-8 explicitamente
I2	A saída do hook com acentos quebrava	<code>json.dumps(..., ensure_ascii=False)</code> em um stdout cp1252	JSON ASCII com escapes <code>\u</code>
I3	O hook ficava mudo quando servido pelo servidor remoto	O servidor remoto rodava uma versão mais antiga que respondia decisões binárias em outro formato; a busca de chave do hook levantava uma exceção que o tratador fail-open engolia	ler o campo de probabilidade presente nos dois formatos; reimplantar o servidor
I4	O classificador binário respondia como decisão de múltipla escolha	formato de resposta único no cliente do classificador	rótulos binários respondem com o tipo de decisão binária
I5	Janela maior que o modelo	1.500 caracteres excedem 256 tokens a cerca de 3,04 caracteres por token: truncamento silencioso	janelas de 640/160; teste de fronteira; 9,2% das janelas ainda passam de 256 tokens

injeção conhecida e um texto benigno conhecido pelo caminho real de implantação, contra cada versão de servidor em uso, teria apanhado os dois. O mesmo vale para o ambiente de treino: uma atualização de biblioteca na imagem do notebook na nuvem removeu um argumento de treino e quebrou a primeira execução de fine-tuning, e uma atualização da biblioteca de datasets removeu os scripts de carga.

Lição 4: meça a janela em tokens. O I5 é um caso de uma armadilha geral: a camada de integração mede texto em caracteres, o modelo em tokens, e a razão depende do gênero do texto. READMEs técnicos mediram uma mediana de 3,04 caracteres por token (10º percentil 2,61), então uma janela de 1.500 caracteres tem cerca de 493 tokens e aproximadamente metade de cada janela era descartada. O script de medição não foi preservado (lacuna L5). As janelas de 640 caracteres escolhidas para caber ainda deixaram 9,2% das janelas acima do limite na execução dos baselines (mediana de 196 tokens, máximo de 646), porque a razão varia dentro de uma página, e não só entre gêneros. As janelas deveriam ser cortadas em tokens; o avaliador publicado ainda as corta em caracteres, como neste estudo, mas conta os tokens de cada janela e relata a fração que passa do contexto do modelo (Seção 13).

Lição 5: avalie pelo caminho de código da implantação. O avaliador de páginas chama a própria função de pontuação do hook. Todos os defeitos acima seriam invisíveis para um avaliador que chamasse o modelo diretamente.

Lição 6: congele as entradas da avaliação e guarde as notas cruas. As páginas benignas foram selecionadas primeiro ao vivo, a partir dos pacotes instalados. Quando a v6 foi avaliada, pacotes novos tinham trocado 21 das 217 páginas e alterado outras 2, e só uma exportação congelada e com impressão digital manteve válida a comparação entre v3 e v6. As notas foram gravadas como probabilidades arredondadas em quatro casas, o que escondeu a ordenação residual da v3 por página (visível só nos log-odds, Seção 5.4) e deixa indefinido o ponto de operação de 2% da v6 (Seção 7.2). Guarde os hashes das entradas e os logits crus.

Depois das correções, o hook tem 9 casos de teste, e as suítes do hook, do classificador e do cache de embeddings (13 testes) passam.

9 Discussão

9.1 O que a acurácia de benchmark mediu aqui

Os números da Tabela 4 foram produzidos com partições fixas, um critério registrado em commit antes da medição, uma trava anti-vazamento a partir da v4 e nenhum ajuste nos conjuntos de teste, com as sobreposições da Seção 3.1 como exceções conhecidas. Mesmo assim, estavam inflados: o melhor placar no Weni dependia de dados de treino que uma descontaminação remove (Seção 4d). E responderam a uma pergunta diferente da que importava. Os benchmarks mediram se o modelo separa injeções de texto benigno majoritariamente *conversacional*, porque era disso que o lado benigno dos testes era feito em grande parte: MASSIVE-pt e Dolly-pt são pedidos curtos a um assistente (o Dolly entra só pelo campo de instrução), os conjuntos em inglês pareciam injeções sobretudo com prompts de tipo conversa, e o Weni não contém nenhum item benigno. O único item benigno técnico escrito na suíte foi uma frase de teste de fumaça, e a partir da rodada A ela estava errada em toda rodada em que sua nota foi gravada (Seção 6.2). Os baselines públicos mostram o mesmo de fora do nosso pipeline: nas frases do xTRam1 o nosso modelo fica em primeiro entre seis, ajudado por dados da mesma distribuição, e por página empata em último (Seção 5.4).

Seguem cinco correções concretas para quem avalia um classificador de injeção para uso por agentes:

1. **O lado benigno do teste precisa ser tirado da distribuição de implantação.** Para um guard de web fetch ou de leitura de arquivos, isso significa documentação, código, configuração, logs e páginas web comuns, não conversa. Os dois modelos públicos que mantêm o aviso falso por página abaixo de 2% no limiar 0,5 são os dois tamanhos do Prompt Guard 2, cujo card relata explicitamente dados benignos da web no treino; só o de 86M detecta também uma parte útil das páginas com injeção (62%, contra 15% no de 22M; Seção 5.4).
2. **Avalie na unidade de decisão.** Se a decisão implantada é “avisar sobre esta página”, a métrica é a taxa de aviso falso por página no limiar implantado, com a detecção medida nas mesmas páginas com um ataque inserido.
3. **Relate intervalos e margens.** Com $n = 300$ o critério de 95% não podia ser distinguido do resultado (Seção 4b); e o efeito de capacidade de 20 pontos entre e5-base e e5-large nos mesmos dados (com vazamento) supera a maioria das diferenças entre as receitas de dados que testamos.
4. **Descontamine contra todos os testes e confira a auditoria com um retreino.** Um corte de cosseno em 0,9 contra o conjunto ouro, aplicado a posteriori, estimou em 1,3 ponto o vazamento no nosso melhor modelo; retreinar sem as duplicatas de todos os testes, as traduções delas e as quase duplicatas do conjunto ouro custou 26,7, e mais nos textos que o corte tinha chamado de limpos (29,8 pontos) do que nos demais (13,8; Seção 4d). Uma auditoria só contra o conjunto ouro não enxerga o que as duplicatas dos outros testes acrescentam, e as similaridades de embeddings são comprimidas, então um limiar que parece rigoroso pode separar pouco. Confira todo teste, inclusive as cópias traduzidas: duplicatas exatas após normalização apareceram em quase todo teste que usamos (Seção 3.1).
5. **Separe os efeitos antes de atribuí-los.** Atribuímos primeiro a queda da v6 no Weni aos negativos novos; um controle descontaminado sem eles caiu quase o mesmo, então a atribuição foi precipitada, embora esse controle também não resolva a questão (Seção 7.2). Uma correção voltada para avisos falsos precisa ser conferida de novo no conjunto ouro, e contra um controle que difira dela só pela correção.

9.2 Modelos de decisão tipados com agentes: um diálogo com Akita (2026)

O post de Akita (Akita, 2026a) dá critérios práticos de quando um modelo de decisão tipado é a ferramenta certa. Concordamos com eles, e o nosso guard os satisfaz no papel: um único julgamento repetido (cada página buscada), uma lista fechada de ações (avisar ou não), código em volta aplicando regras rígidas (o hook) e erros que podem seguir para conferência (o agente e, em última instância, o usuário decidem). O guard também seguiu o conselho, dado no ensaio dentro do post, de não confiar só a um modelo desses a liberação de operações destrutivas: ele era só de aviso e fail-open por desenho. Que um desenho que atende a todos esses critérios ainda assim tenha falhado por completo é, achamos, a contribuição útil. Especificamente:

Recalibrar com os próprios exemplos é necessário, mas não suficiente. O ensaio do post mostra um modelo hospedado que se diz muito seguro mesmo sem informação nenhuma, e recomenda refazer a calibração com casos rotulados do próprio usuário. Os nossos modelos com fine-tuning tinham o defeito análogo (saídas não calibradas e saturadas). Mas recalibrar no *nosso próprio split de validação* não teria ajudado, porque esse split vinha da mesma distribuição conversacional. Os exemplos usados na calibração precisam ser tirados das entradas que o modelo verá em uso, e, para um guard de conteúdo, esses não são os exemplos que se tem naturalmente à mão.

Treinar um classificador dedicado exige definir a tarefa pela sua distribuição de entrada. O post anterior aponta um classificador dedicado à tarefa como uma alternativa a uma API de decisão hospedada (Akita, 2026b). Fizemos exatamente isso, e o classificador era excelente na tarefa tal como definida pelos seus rótulos de treino. A tarefa tal como definida pelas suas entradas (conteúdo buscado arbitrário, em sua maioria técnico e benigno) nunca foi representada. Para modelos de decisão em geral, a pergunta a fazer não é só “quais são as opções?”, mas também “como é a classe negativa em uso?”.

O escalonamento por confiança colapsa sob saturação. Um padrão comum de integração encaminha decisões de alta confiança para automação e as de baixa confiança para um modelo mais forte ou para um humano. Com as probabilidades cravadas em 1,0, a faixa de escalonamento fica vazia: toda página é um caso “fácil”, confiante e errado. A v6 mostra a imagem espelhada: 23% dos ataques do Weni recebem $P = 0,0000$, confiantes e errados na outra direção (4,3% na v3-pub). O roteamento por confiança só é tão bom quanto a calibração *sob a distribuição de implantação*.

Corrigir a classe negativa pode mover a fronteira também para a classe positiva, mas confira antes de concluir que moveu. Lemos primeiro a perda de 31 pontos da v6 no Weni como o preço do conserto: ataques formulados como tarefas comuns ficando indistinguíveis de instruções técnicas. O controle descontaminado (Seção 7.2) perde quase o mesmo sem nenhum negativo novo: no limiar 0,5 a v6 fica 4,3 pontos abaixo dele, dentro do ruído, enquanto na ordenação contra negativos conversacionais a diferença é maior (AUC 0,977 contra 0,883); quanto da perda os negativos causam continua em aberto. Quando as duas classes compartilham a forma de superfície, mais dados de um lado podem deslocar a fronteira através do outro, e uma interface tipada não avisa o integrador quando isso acontece; mas uma queda num conjunto ouro só é evidência de fronteira deslocada depois de descartado o vazamento.

Falha silenciosa é propriedade da integração, não do modelo. As evidências do post dizem respeito ao comportamento do modelo; o nosso bug mais perigoso (I3) não tinha nada a ver com o modelo. Um guard que falha aberto e em silêncio parece exatamente um guard que não encontra nada.

9.3 Relação com trabalhos concorrentes

Liu (2026) reavalia quinze detectores públicos em benchmarks de agentes (AgentDojo, τ -bench, BIPIA) e em texto benigno que inclui cerca de 2.500 trechos de READMEs do GitHub, de 80 a 220 palavras. O estudo relata que o classificador da deepset marca todos os trechos de README e é inutilizável, que tirar a marcação HTML dos READMEs corta pela metade ou mais a taxa de falso positivo de dois outros detectores, e que os falsos positivos em texto comum não preveem os falsos positivos em saídas de ferramentas. Suas recomendações são medir falsos positivos nas entradas do próprio agente, relatar a detecção numa taxa fixa e baixa de falso positivo, verificar se o detector foi treinado no benchmark e preferir detectores treinados com entradas parecidas com as que vão filtrar. Ele apareceu em 2 de outubro, com o nosso estudo em andamento, e só o encontramos durante a revisão da literatura; os dois foram feitos de forma independente.

Lemos o nosso estudo como uma replicação independente dessa lacuna, com três diferenças. Primeiro, outra língua e um detector treinado por nós. Os seus dados de treino são conhecidos, então a lacuna pôde ser rastreada até a composição do lado benigno e o diagnóstico pôde ser testado por retreino (Seção 7), o que um estudo de modelos de terceiros não consegue fazer. Onde os dois estudos se sobrepõem, eles concordam: o modelo da deepset avisa em toda página no nosso protocolo, como em todo trecho no de Liu. O nosso resultado de vazamento é também um caso concreto da terceira recomendação de Liu, verificar se o detector foi treinado no benchmark: o nosso próprio modelo, com uma auditoria de vazamento no pipeline, tinha sido treinado com duplicatas e vizinhos próximos dos seus testes (Seção 4d). Segundo, a unidade de avaliação: pontuamos páginas inteiras pelo caminho de código de um hook implantado, o que expõe a amplificação sobre as janelas (Seção 5.3), a troca ligada ao tamanho da janela (Seção 5.4) e os defeitos de integração da Seção 8. Terceiro, o enquadramento: o nosso classificador foi implantado como uma decisão tipada, e a Seção 9.2 tira as consequências para essa família de modelos.

O resultado de Liu sobre marcação toca diretamente o nosso. As janelas que a v6 ainda marca são badges, links de patrocínio e tabelas markdown (Seção 7.2), e as quatro páginas benignas em que o Prompt Guard 2 (86M) avisa também são trechos fora da prosa (uma tabela de marcas de verificação, badges, uma transcrição de terminal, uma tabela de opções; Seção 6.3). As duas observações são coerentes com a marcação e outros conteúdos fora da prosa, e não a prosa imperativa, serem a principal fonte restante de avisos falsos uma vez coberto o texto técnico. Não medimos os nossos modelos com e sem marcação. Fazer isso, e acrescentar como negativos boilerplate de README tirado de uma fonte disjunta do benchmark, é o próximo experimento.

9.4 Sobre resultados negativos

O guard fica fora do hook de web fetch; a v3 continua no kit local apenas para texto conversacional, e o kit segue com outras decisões. Relatamos a falha em detalhe porque o caminho que levou a ela (dados abertos, splits cuidadosos, um critério fixado de antemão, um resultado forte no benchmark) é o caminho que a maioria dos praticantes seguiria, e o passo que apanhou o problema, avaliar na unidade de uso em conteúdo comum, é barato e ainda raramente relatado; uma exceção recente mede detectores em trechos benignos de READMEs em inglês, e não em páginas inteiras (Liu, 2026). A tentativa de correção e o retreino descontaminado são relatados pela mesma razão: a primeira funcionou no que mirava, e o segundo mostrou que uma queda quase do mesmo tamanho acontece sem a correção quando os dados vazados saem do treino, de modo que o preço que atribuímos a ela não está estabelecido.

10 Limitações

- **Uma única execução por configuração.** Não se mediu variância entre sementes; diferenças de poucos pontos entre rodadas (por exemplo, v3 contra v5, ou v3-pub contra v6 no Weni) podem ser ruído. A diferença pareada de 26,7 pontos entre v3 e v3-pub é grande para um efeito de semente, mas isso não foi medido.
- **Vazamento no melhor modelo.** A v3 foi treinada antes da trava anti-vazamento; 115 dos seus exemplos de treino são quase duplicatas de textos do Weni. Um corte a posteriori em cosseno 0,9 estimou o efeito deles em 1,3 ponto; o retreino descontaminado perde 26,7, mais nos textos sem um vizinho desses (29,8 pontos) do que nos que têm um (13,8; Seção 4d). Qual das remoções da v3-pub responde pelo efeito, e quanto dele é vazamento, e não dado de ataque perdido, não está isolado: ela também removeu 967 exemplos de ataque, 513 deles duplicatas exatas de outros testes, e o controle “v3 menos as 115” não foi rodado. A divisão por fonte dos 4.008 exemplos removidos na v4 não foi gravada, e a contabilidade da Seção 4(c) é uma inferência entre exportações (lacuna L1).
- **O xTRam1 não está fora da distribuição** da v2 à v6, e duplicatas exatas de itens de teste estavam nos dados de treino da v3 em quase todo teste, entre elas 20 dos 262 itens do jackhhao e 83 dos 1.500 do xTRam1 (Seção 3.1). Só a v3-pub está livre delas; no xTRam1 ela faz 95,4%, contra 98,7% da v3.
- **Avaliação incompleta da v3.** A v3 só foi medida no deepset, no jackhhao e nos primeiros 300 itens do SPML na comparação da v6, e nunca no rikka, ShieldLM ou yanis; números de alguns desses que aparecem nos metadados do pack implantado não têm origem rastreável e não são usados aqui.
- **Intervalos.** Os intervalos de Wilson são relatados para o Weni, para as taxas por página, para as páginas da comparação entre v3 e v6 e para as linhas da Tabela 5; os demais testes das Tabelas 4 e 10 não têm intervalos no texto (lacuna L6; o v3pub.json os traz).
- **Idioma.** O conjunto benigno em português (MASSIVE) é português europeu; as traduções automáticas não foram revisadas por um humano.
- **Procedência do Weni.** Que o Weni seja material traduzido do HackAPrompt tem apenas sustentação indireta (Seções 3.4 e 4c). Uma análise de erros registrada da v2 no Weni existe apenas em uma mensagem de commit, sem script nem contagens (lacuna L7).
- **Escopo do benchmark de páginas.** As páginas benignas são de um único gênero (READMEs de pacotes Python), selecionadas entre os pacotes instalados em um ambiente; as páginas com injeção reutilizam os mesmos READMEs, então a detecção não é informativa enquanto todo README dispara. As injeções são texto colado, não marcação escondida, e, com 60 páginas com injeção, os intervalos da detecção cobrem cerca de ± 12 pontos. A execução original não gravou as taxas por janela (a dos baselines gravou) nem qual instância do servidor respondeu (lacuna L4).
- **Medições não verificadas.** O número de caracteres por token (lacuna L5), o ganho de velocidade da tradução (só mensagem de commit), o uso efetivo das duas GPUs e o tamanho de batch efetivo (lacuna L10) não foram verificados de forma independente.
- **Revisões dos dados.** Os conjuntos foram carregados sem revisões fixadas; as revisões registradas nos artefatos foram capturadas depois das execuções.

- **Avisos dos data cards.** O card do SPML desaconselha treinar detectores com ele; nós o usamos e apontamos isso como ameaça à validade.
- **Afirmações causais testadas só em parte.** A v6 mudou três coisas ao mesmo tempo (negativos técnicos, remoção das 115 quase duplicatas do Weni, equilíbrio de classes) numa única execução. A v3-pub descontaminada é um controle parcial, mas as bases não são idênticas: a v6 mantém as duplicatas dos testes, as traduções delas e as linhas do ShieldLM de origem excluída que a v3-pub remove, e a v3-pub remove 1.758 exemplos onde a v6 remove 115 (Seção 7.1). A causa da queda da v6 no Weni continua, portanto, em aberto até o controle “v3 menos as 115” ser rodado. Os testes técnicos da v6 estão dentro da distribuição dela, e não medimos o efeito da marcação (Seções 7.2 e 9.3).
- **Resolução das notas.** A nossa avaliação gravou probabilidades arredondadas em quatro casas, o que limita a análise de limiar e de AUROC perto de 0 e de 1 (Seção 7.2); só a execução dos baselines gravou log-odds.
- **Baselines.** O Prompt Guard v1 não foi avaliado (acesso restrito). Os dados de treino dos modelos públicos não estão documentados por inteiro e podem se sobrepor ao Weni (via HackAPrompt) ou ao xTRam1. O Prompt Guard 2 só marca a tentativa explícita de passar por cima de instruções, uma noção de ataque mais estreita que a dos conjuntos de teste, de modo que a sua detecção mede também essa diferença, e não só erros. As latências vêm de uma única VM ARM de 4 núcleos, exceto as do Prompt Guard 2, medidas em instâncias x86 de notebook e convertidas por uma âncora na mesma instância cuja razão variou de 1,18 a 1,82 com a janela e o lote, de modo que só indicam a ordem de grandeza.
- **Truncamento.** Todos os nossos modelos leem no máximo 256 tokens por janela, e 9,2% das janelas de 640 caracteres passam disso (os baselines públicos da Seção 5.4 leem 512).

11 Conclusão

Um classificador de prompt injection em português do Brasil, treinado só com dados abertos, passou nos seus benchmarks de frases, perdeu por um texto o critério do conjunto ouro fixado de antemão e deu aviso falso em toda página de documentação técnica que lhe foi mostrada. As duas metades desse resultado se mostraram importantes. O número do benchmark estava inflado: retreinada sem duplicatas nem quase duplicatas dos testes, a mesma receita detecta 68,0% dos ataques nativos em pt-BR, e não 94,7%. Uma auditoria por similaridade de cosseno tinha estimado a inflação em 1,3 ponto; a queda de 26,7 pontos a limita por cima, já que o modelo retreinado também tem menos exemplos de ataque. A falha em uso é real e separada: o modelo descontaminado continua avisando em toda página. Ela não é exclusiva do nosso modelo, já que um classificador público faz o mesmo, e a unidade de avaliação muda o veredito: os benchmarks por frase e por página ordenaram os seis modelos em sentidos opostos. Só um modelo público, o Llama Prompt Guard 2 (86M), fica abaixo de 2% de aviso falso por página no limiar 0,5 com detecção útil (1,8%, com 62% de detecção; o irmão de 22M não dá aviso falso, mas detecta 15%). O card dos dois relata dados benignos da web no treino, e ainda assim o de 86M detecta só 30% das injeções curtas e não atinge o nosso critério por frase no Weni, de modo que estreita o problema sem fechá-lo. Acrescentar texto benigno técnico eliminou os avisos falsos em frases técnicas e a maior parte deles nas páginas; contra o controle descontaminado, custou 4,3 pontos de detecção no Weni no limiar 0,5, dentro do ruído, de modo que a queda que atribuíamos a ele não pode ser cobrada dele só pela comparação com a v3; um controle que isole os vizinhos do Weni diria quanto dela é da correção. Nenhuma versão que treinamos atende aos dois critérios. As conclusões práticas são modestas: descontaminar contra

todos os testes e conferir a auditoria com um retreino; tirar o lado benigno do teste da distribuição de implantação; avaliar na unidade de decisão pelo caminho de código da implantação; congelar as entradas e guardar as notas cruas; e separar os efeitos antes de atribuí-los. O hook continua desligado.

12 Ética e dados

Todos os dados de treino e de avaliação são públicos. As licenças foram lidas da API do Hugging Face Hub, dos dataset cards e dos arquivos LICENSE/NOTICE em 6 de outubro de 2026. Dois conjuntos de avaliação (Weni e xTRam1) não declaram licença; nós os carregamos só para avaliação, baixamos em tempo de execução e não os redistribuímos, nem as suas traduções. Porém, 577 linhas do split de treino do xTRam1 chegaram indiretamente aos nossos dados de treino, pelo ShieldLM (Seção 3.1); os pesos da v3-pub, treinada em 7 de outubro, as excluem, junto com as linhas do ShieldLM cuja licença difere entre a origem e o agregador (Seção 3.4). O conteúdo do Dolly e do Stack Overflow é CC-BY-SA; os pesos da v3-pub, treinada com o Dolly e sem o Stack Overflow, são publicados sob MIT seguindo o precedente de modelos treinados nos mesmos dados, e declaramos essa posição explicitamente. O Llama Prompt Guard 2 foi avaliado sob a Llama 4 Community License apenas como baseline: as suas saídas não foram usadas para treinar, calibrar ou destilar os nossos modelos, e os seus pesos não são redistribuídos (a cópia usada para rodá-lo fica num armazenamento privado do serviço de notebooks).

O benchmark de páginas não redistribui nenhum README: ele distribui nomes de pacotes, versões e hashes, e um construtor baixa as páginas do PyPI e as confere. Nenhum dado pessoal, texto privado ou dado organizacional foi usado neste estudo. O modelo publicado detecta injeções e não é uma ferramenta de ataque; não publicamos texto de ataque além do que os conjuntos públicos citados já contêm.

Um guard com taxa de aviso falso de 100% é nocivo de um jeito sutil: os usuários aprendem a ignorar seus avisos. O model card abre com a falha por página e relata os números da própria v3-pub; os 94,7% da v3, inflados por vazamento, aparecem nele só como comparação.

Uso de assistência de IA. Boa parte do código experimental, dos logs e do primeiro rascunho deste artigo foi produzida com a ajuda de um agente de programação baseado em LLM. Todo número relatado é rastreado até uma linha de log, um arquivo de resultado ou um commit; as afirmações que não puderam ser rastreadas estão listadas como limitações.

13 Artefatos

O código, os resultados e o benchmark de páginas estão publicados em <https://github.com/alucardigo/passing-the-benchmark>; os pesos ajustados são publicados à parte, no Kaggle Models.

- **Código** (Apache-2.0): pipeline de treino e exportação, tradução determinística com cache, ensemble de pesos fixos, construtor e avaliador do benchmark de páginas, um hook de agente de referência (desligado por padrão), testes. Repositório: <https://github.com/alucardigo/passing-the-benchmark>. O repositório público começa sem o histórico de desenvolvimento.
- **Pesos** (MIT): a v3-pub (Seção 3.4, Tabela 5) e a sonda da rodada B, publicadas no Kaggle Models como passing-the-benchmark-guard. O card abre com a falha por página e relata os números da própria v3-pub (Weni 68,0%, xTRam1 95,4%, 217 de 217 avisos falsos por página). A v6 não atende ao critério por página (Seção 7.2) e não é publicada como guard.

- **Resultados** (CC-BY-4.0): um JSON por rodada com o SHA-256 dos pesos e as probabilidades por página indexadas, as probabilidades por item das comparações entre v3 e v6 e com a v3-pub, e os log-odds por página e por frase da execução dos baselines, sem texto de terceiros.
- **Benchmark de páginas**: um manifesto de 217 páginas benignas e 60 inserções (pacote, versão, SHA-256 dos metadados e da página; fonte, revisão, md5 e posição de cada inserção) mais um construtor que recusa hashes divergentes, seguindo o padrão do Decision Index. O avaliador aceita qualquer classificador do Hugging Face, corta as janelas em caracteres, conta os tokens delas e relata a fração que passa do contexto do modelo, além dos avisos falsos e da detecção por limiar com intervalos de Wilson.
- **Arquivo de atribuição** listando cada conjunto de dados, modelo e revisão com a sua licença.

Referências

- Fabio Akita. Entendendo o hype do Jev: pra que serve? AkitaOnRails blog, 2026-10-03, October 2026a. URL <https://akitaonrails.com/2026/10/03/entendendo-o-hype-do-jev-pra-que-serve/>. In Portuguese; includes an essay by Paulo Câmara written for the post (he discloses that he co-founded a company competing in the same market).
- Fabio Akita. Por que coisas como TypeSafe IA não me interessam. AkitaOnRails blog, 2026-09-16, September 2026b. URL <https://akitaonrails.com/2026/09/16/por-que-coisas-como-typesafe-ia-nao-me-interessam/>. In Portuguese.
- Diogo Almeida. Introducing system one models & Jev. TypeSafe AI blog, 2026-09-15, September 2026. URL <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. The FAQ states that the “System One” name draws on Kahneman’s System 1 / System 2 distinction.
- apolinario. Decision index: reproduction kit for the typed-decision benchmark suite. GitHub repository <https://github.com/apolinario/decision-index>; leaderboard data in the Hugging Face Space `multimodalart/jev-decision-index` (file `data/index-v0.2.1.json`, generated 2026-09-28), 2026. URL <https://github.com/apolinario/decision-index>. Version 0.2.1 (2026-09-27); MIT (code); chance-corrected skill score, unanswered requests count as wrong.
- Michelle Chen, Alex Reneau, and Kevin Flansburg. Introducing Clef: our open-source decision models, and new RL fine-tuning platform. Cloudflare blog, 2026-10-01, October 2026. URL <https://blog.cloudflare.com/clef-decision-models/>. Clef (frozen Qwen3.8-27B) and Clef-flash (frozen Qwen3.5-9B) with LoRA adapters and a scoring head; weights on Hugging Face under Apache-2.0.
- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400. USENIX Association, 2025. ISBN 978-1-939133-52-6. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>. Preprint first posted February 2024.
- Cloudflare. Clef-flash. Hugging Face model `Cloudflare/clef-flash`, 2026. URL <https://huggingface.co/Cloudflare/clef-flash>. Apache-2.0; based on Qwen/Qwen3.5-9B; revision 17f0b0ad.
- Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free Dolly: Introducing the world’s first truly open instruction-tuned LLM. Databricks blog, 2023-04-12, April 2023. URL <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>. Dataset `databricks/databricks-dolly-15k`, CC-BY-SA-3.0, revision bdd27f4d.
- Convai Innovations. Laya: a non-autoregressive typed-decision engine. GitHub repository <https://github.com/NandhaKishorM/laya>; Hugging Face models `convaiinnovations/laya` and

- convaaiinnovations/laya-multilingual, 2026. URL <https://github.com/NandhaKishorM/laya>. Apache-2.0; English checkpoint on ModernBERT-large (421M), multilingual checkpoint on mmBERT-base (322M); PyPI package `laya`.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, pages 82895–82920, 2024. doi: 10.52202/079017-2636. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.
- Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL <https://arxiv.org/abs/2503.18813>.
- deepset. deberta-v3-base-injection. Hugging Face model `deepset/deberta-v3-base-injection`, 2023a. URL <https://huggingface.co/deepset/deberta-v3-base-injection>. MIT; fine-tuned from microsoft/deberta-v3-base on deepset/prompt-injections; revision 80dda00d.
- deepset. prompt-injections. Hugging Face dataset `deepset/prompt-injections`, 2023b. URL <https://huggingface.co/datasets/deepset/prompt-injections>. Apache-2.0; 546 train / 116 test; English and German; revision 4f61ecb0.
- dmilush. shieldlm-prompt-injection. Hugging Face dataset `dmilush/shieldlm-prompt-injection`, 2026. URL <https://huggingface.co/datasets/dmilush/shieldlm-prompt-injection>. MIT (curation; sources keep their own licenses); revision dd660ae7.
- Jack FitzGerald, Christopher Hench, Charith Peris, Scott Mackie, Kay Rottmann, Ana Sanchez, Aaron Nash, Liam Urbach, Vishesh Kakarala, Richa Singh, Swetha Ranganath, Laurie Crist, Misha Britan, Wouter Leeuwis, Gokhan Tur, and Prem Natarajan. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4277–4302. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.235. Preprint posted April 2022; data: <https://huggingface.co/datasets/AmazonScience/massive> (CC-BY-4.0), used via the mirror `mteb/amazon_massive_intent` revision 940fd47a.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, pages 79–90. ACM, 2023. doi: 10.1145/3605764.3623985.
- Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In Marilyn Walker, Heng Ji, and Amanda Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 107–112, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-2017. URL <https://aclanthology.org/N18-2017/>.
- Helsinki-NLP. opus-mt-tc-big-en-pt. Hugging Face model `Helsinki-NLP/opus-mt-tc-big-en-pt`, 2022. URL <https://huggingface.co/Helsinki-NLP/opus-mt-tc-big-en-pt>. CC-BY-4.0; transformer-big, target tokens `>>por<<` and `>>pob<<`; revision 9f2863d8.
- Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting, 2024. URL <https://arxiv.org/abs/2403.14720>.
- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search, 2019. URL <https://arxiv.org/abs/1909.09436>.

- imoxto. `prompt_injection_hackaprompt_gpt35`. Hugging Face dataset `imoxto/prompt_injection_hackaprompt_gpt35`, 2023. URL https://huggingface.co/datasets/imoxto/prompt_injection_hackaprompt_gpt35. Mirror of HackAPrompt (gpt-3.5-turbo); no license field in the card metadata; revision efc8f438.
- intfloat. `multilingual-e5-large`. Hugging Face model `intfloat/multilingual-e5-large` (and `multilingual-e5-base`), 2023. URL <https://huggingface.co/intfloat/multilingual-e5-large>. MIT; revisions 3d7cfbda (large) and d1287505 (base); 559,890,946 and 278,044,162 parameters per safetensors metadata.
- jackhhao. `jailbreak-classification`. Hugging Face dataset `jackhhao/jailbreak-classification`, 2023. URL <https://huggingface.co/datasets/jackhhao/jailbreak-classification>. Apache-2.0; revision 2f2ceeb3.
- Dennis Jacob, Hend Alzahrani, Zhanhao Hu, Basel Alomair, and David Wagner. PromptShield: Deployable detection for prompt injection attacks. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (CODASPY '25)*, pages 341–352. ACM, 2025. doi: 10.1145/3714393.3726501.
- Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011. ISBN 978-0-374-27563-1.
- Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, Tony Lee, Etienne David, Ian Stavness, Wei Guo, Berton Earnshaw, Imran Haque, Sara M. Beery, Jure Leskovec, Anshul Kundaje, Emma Pierson, Sergey Levine, Chelsea Finn, and Percy Liang. WILDS: A benchmark of in-the-wild distribution shifts. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5637–5664. PMLR, 2021. URL <https://proceedings.mlr.press/v139/koh21a.html>.
- koutch. `stackoverflow_python`. Hugging Face dataset `koutch/stackoverflow_python`, 2023. URL https://huggingface.co/datasets/koutch/stackoverflow_python. Stack Overflow content, CC-BY-SA; no license field in the card metadata; revision ee6f3955.
- Hao Li and Xiaogeng Liu. InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models, 2024. URL <https://arxiv.org/abs/2410.22770>. Preprint of PIGuard (li2025piguard); introduces the NotInject benchmark (339 benign sentences with trigger words).
- Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao. PIGuard: Prompt injection guardrail via mitigating overdefense for free. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30420–30437, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1468. URL <https://aclanthology.org/2025.acl-long.1468/>.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847. USENIX Association, August 2024. ISBN 978-1-939133-44-1. URL <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>.
- Zhuowen Liu. Passing the test you trained on: Re-evaluating prompt-injection detectors for LLM agents, 2026. URL <https://arxiv.org/abs/2610.03448>. Submitted 2026-10-02; code: <https://github.com/lzwhehe/benign-instruction-bench>.
- R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3428–3448, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1334. URL <https://aclanthology.org/P19-1334/>.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- Meta. Prompt guard 86m. Hugging Face model `meta-llama/Prompt-Guard-86M`, 2024. URL

<https://huggingface.co/meta-llama/Prompt-Guard-86M>. Llama 3.1 license; first released July 2024.

Meta. Llama prompt guard 2. Hugging Face models `meta-llama/Llama-Prompt-Guard-2-86M` and `-22M`; model card in <https://github.com/meta-llama/PurpleLlama>, 2025. URL <https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M>. Llama 4 Community License; 86M on mDeBERTa-base, 22M on DeBERTa-xsmall; 512-token window.

Yanis Miraoui. `prompt_injections`. Hugging Face dataset `yanismiraoui/prompt_injections`, 2023. URL https://huggingface.co/datasets/yanismiraoui/prompt_injections. Apache-2.0; 7 languages incl. Portuguese; revision `bd55359f`.

Nan-Do. `code-search-net-python`. Hugging Face dataset `Nan-Do/code-search-net-python`, 2023. URL <https://huggingface.co/datasets/Nan-Do/code-search-net-python>. Python subset of CodeSearchNet (husain2019codesearchnet); revision `39db9186`.

pacovaldez. `stackoverflow-questions`. Hugging Face dataset `pacovaldez/stackoverflow-questions`, 2022. URL <https://huggingface.co/datasets/pacovaldez/stackoverflow-questions>. Stack Overflow content, CC-BY-SA (card metadata says Apache-2.0); revision `869802e5`.

Jared Palmer. `kev-9b`. Hugging Face model `jaredpalmer/kev-9b`, 2026. URL <https://huggingface.co/jaredpalmer/kev-9b>. Apache-2.0; LoRA adapter and head over Qwen/Qwen3.5-9B-Base; revision `db029f08`.

Fábio Pérez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models, 2022. URL <https://arxiv.org/abs/2211.09527>. ML Safety Workshop, NeurIPS 2022.

ProtectAI.com. Fine-tuned DeBERTa-v3-base for prompt injection detection. Hugging Face model `protectai/deberta-v3-base-prompt-injection-v2`, 2024. URL <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>. Apache-2.0; English only; revision `90c9989b`; citation as given on the model card.

Proventra. `mdeberta-v3-base-prompt-injection`. Hugging Face model `proventra/mdeberta-v3-base-prompt-injection`, 2025. URL <https://huggingface.co/proventra/mdeberta-v3-base-prompt-injection>. MIT; fine-tuned from microsoft/mdeberta-v3-base; revision `b8a89d30` on 2026-10-06.

Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet? In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5389–5400. PMLR, 2019. URL <https://proceedings.mlr.press/v97/recht19a.html>.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.442. URL <https://aclanthology.org/2020.acl-main.442/>.

rikka-snow. `prompt-injection-multilingual`. Hugging Face dataset `rikka-snow/prompt-injection-multilingual`, 2024. URL <https://huggingface.co/datasets/rikka-snow/prompt-injection-multilingual>. MIT; 11 languages incl. Portuguese; revision `flad1f3d`.

Sander Schulhoff, Jeremy Pinto, Ansum Khan, Louis-François Bouchard, Chenglei Si, Svetlana Anati, Valen Tagliabue, Anson Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4945–4977, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.302. URL <https://aclanthology.org/2023.emnlp-main.302/>.

Reshabh K. Sharma. `SPML_Chatbot_Prompt_Injection`. Hugging Face dataset `reshabhs/SPML_Chatbot_Prompt_Injection`, 2024. URL https://huggingface.co/datasets/reshabhs/SPML_Chatbot_Prompt_Injection. MIT; revision `02ce8084`; accompanies the SPML paper (sharma2024spml).

- Reshabh K. Sharma, Vinayak Gupta, and Dan Grossman. SPML: A DSL for defending language models against prompt attacks, 2024. URL <https://arxiv.org/abs/2402.11755>.
- Yusuf Khalid Shire and Sang-Chul Kim. PIDS-Bench: Evaluating prompt-injection detectors under over-defense, obfuscation, and distribution shift. *IEEE Access*, 14:134184–134205, 2026. doi: 10.1109/ACCESS.2026.3728186.
- Jörg Tiedemann. The Tatoeba translation challenge – realistic data sets for low resource and multilingual MT. In *Proceedings of the Fifth Conference on Machine Translation*, pages 1174–1182, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.wmt-1.139. URL <https://aclanthology.org/2020.wmt-1.139/>.
- Jörg Tiedemann and Santhosh Thottingal. OPUS-MT – building open translation services for the world. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, pages 479–480, Lisboa, Portugal, November 2020. European Association for Machine Translation. URL <https://aclanthology.org/2020.eamt-1.61/>.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings: A technical report, 2024. URL <https://arxiv.org/abs/2402.05672>.
- Weni. prompt-injections-1.0.0. Hugging Face dataset Weni/prompt-injections-1.0.0, 2024. URL <https://huggingface.co/datasets/Weni/prompt-injections-1.0.0>. No license declared; 300 Portuguese attack prompts; revision 9707fa47.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- xTRam1. safe-guard-prompt-injection. Hugging Face dataset xTRam1/safe-guard-prompt-injection, 2024. URL <https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection>. No license declared; synthetic, generated with GPT-3.5-turbo; 8,240 train / 2,060 test; revision a3a877d6.
- Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1*, pages 1809–1820. ACM, 2025. doi: 10.1145/3690624.3709179. Preprint first posted December 2023 (BIPIA).
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.624.

A Computação e custo

Recurso	Medição
Notebook na nuvem, 2× T4, sessões de fine-tuning	v2 1.995,9 s (inclui uma tentativa fracassada de e5-large) · v3 8.511,4 s · v4 1.848,3 s · v5 8.120,5 s · total 20.476 s $\approx$ 5,69 h
Notebook na nuvem, fine-tuning da v6	12.713 s $\approx$ 3,53 h (a primeira tentativa, idêntica, terminou em erro sem log)
Notebook na nuvem, fine-tuning da v3-pub	7.056 s $\approx$ 1,96 h
Cota gratuita de GPU do serviço de notebooks	cerca de 30 h por semana
Embeddings em CPU para as sondas	2.901 s (rodada 0) + 2.777 s (A) + 4.917 s (B) = 10.595 s $\approx$ 2,94 h
VM ARM na nuvem (4 OCPU, aarch64), inferência do e5-large	cerca de 19,6 textos por minuto
VM ARM na nuvem, protocolo de página (ociosa, 4 threads)	por janela cheia: 1.045 ms (e5-large, 256 tokens), 845–879 ms (DeBERTa-base, 512 tokens); cerca de 11 s e 4 s por página média
Pesos (fp16, zipados)	e5-base 515.813.767 B; e5-large 1.031.696.719 B

B Registro de incidentes

#	Incidente	Causa raiz	Correção
I6	A primeira execução de fine-tuning falhou	a atualização de biblioteca da imagem do notebook removeu <code>warmup_ratio</code>	<code>warmup_steps</code> = 6% dos passos
I7	e5-large sem memória na T4	treinado depois do e5-base na mesma sessão; memória fragmentada com batch 16×2	e5-large sozinho, batch 8×4 , gradient checkpointing, expandable segments
I8	Tradução com cerca de 7 s por instrução curta	decodificação gulosa em laço até um teto fixo de 512 tokens	teto em $1,6 \times$ o comprimento da entrada
I9	Traduções perdidas	o processo morto perdeu o lote em buffer	gravação em disco a cada lote
I10	A avaliação local caiu (OpenBLAS, exit 139)	pressão de memória na estação de trabalho	BLAS em thread única no servidor; fallback para a VM ARM
I11	A tradução de e-mails na ARM levou mais de uma hora para 16 textos	geração longa em CPU ARM	tetos de caracteres e de exemplos; nada de geração na ARM
I12	Cada rodada recalculava todos os embeddings	sem cache	cache SHA-1 $\rightarrow$ vetor, só dados públicos
I13	Um conjunto com script de carga não abria	o <code>datasets</code> 4 removeu os scripts de carga	baixar o arquivo de dados diretamente
I14	Ataques fracassados virariam benignos	o HackAPrompt só tem ataques; rótulo implícito	manter só os ataques bem-sucedidos
I15	Servidor remoto declarado morto	timeout de health check de 2 s sobre uma ida e volta lenta pela VPN	timeout de 10 s para servidores remotos
I16	Batches de comprimentos misturados pagavam cerca de $3 \times$ em padding	sem ordenação por comprimento	ordenar por comprimento no classificador
I17	Script de instalação quebrado na VM Linux	finais de linha CRLF exportados pelo git	<code>.gitattributes</code> com LF para scripts

C Linha do tempo

Data (2026, UTC−3)	Evento
1 out	Primeira sonda: xTRam1 86,3%, 0/5 injeções de fumaça em português detectadas
2 out	Pipeline de tradução local; critério final registrado em commit (Weni $\geq 95\%$, xTRam1-pt $\geq 90\%$)
5 out	Rodada A (xTRam1-pt 81,8%); rodada B (Weni 72,3%); v2 (Weni 74,3%); trava anti-vazamento; hook pronto, mas desligado
6 out	v4 (Weni 66,7%); v5 (89,7%); v3 reavaliada (94,67%, depois mostrada dependente de dados de treino vazados); mistura (91,3%); v3 escolhida, status experimental
6 out	Defeitos do hook encontrados; avaliação por página: 100% de avisos falsos nos dois tamanhos de janela; guard rejeitado para web fetch
6 out	Iniciada a exportação da v6 com dados benignos técnicos
6 out	Baselines públicos no protocolo de página; v6 treinada e avaliada contra a v3 nas páginas congeladas; hook mantido desligado
7 out	Llama Prompt Guard 2 (86M, 22M) no protocolo de página; v3-pub treinada e avaliada contra a v3 e a v6 (Weni 68,0%; 100% de avisos falsos por página)